



AWS Academy Cloud Developing
Module 05 Student Guide
Version 2.0.6

200-ACCDEV-20-EN-SG

© 2024, Amazon Web Services, Inc. or its affiliates. All rights reserved.

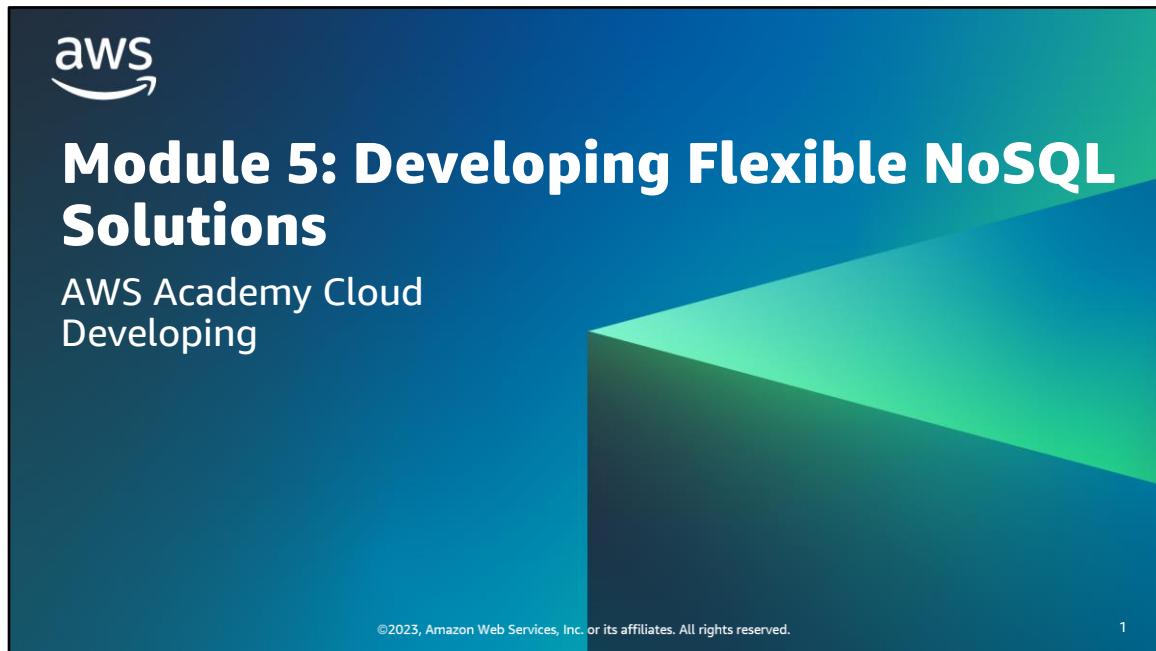
This work may not be reproduced or redistributed, in whole or in part,
without prior written permission from Amazon Web Services, Inc.
Commercial copying, lending, or selling is prohibited.

All trademarks are the property of their owners.

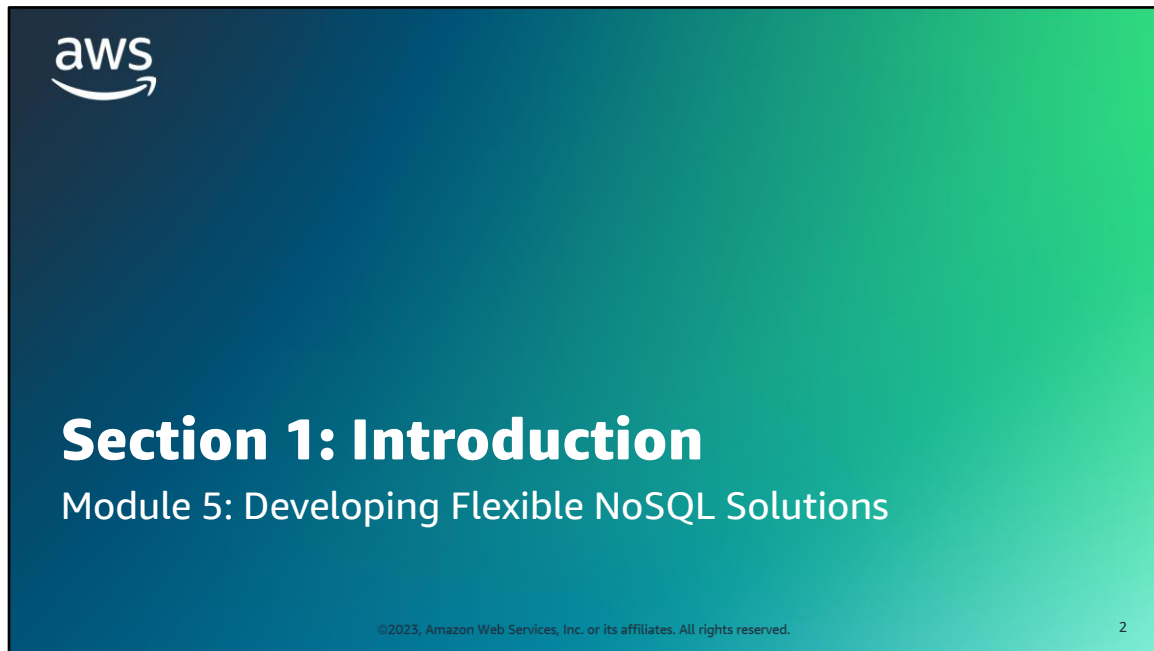
Contents

[Module 5: Developing Flexible NoSQL Solutions](#)

4



Welcome to Module 5: Developing Flexible NoSQL Solutions.



Section 1: Introduction.

Module objectives



At the end of this module, you should be able to do the following:

- Recognize different database options on Amazon Web Services (AWS)
- Identify the features of Amazon DynamoDB
- Describe the components of DynamoDB
- Explain how DynamoDB uses partitions
- Indicate how indexes are used with DynamoDB
- Describe how DynamoDB keeps data consistent
- Recognize when streaming and global tables are used
- Explain the backup and restore process
- Develop with DynamoDB by using the AWS software development kits (SDK)

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

3

At the end of this module, you should be able to do the following:

- Recognize different database options on Amazon Web Services (AWS)
- Identify the features of Amazon DynamoDB
- Describe the components of DynamoDB
- Explain how DynamoDB uses partitions
- Indicate how indexes are used with DynamoDB
- Describe how DynamoDB keeps data consistent
- Recognize when streaming and global tables are used
- Explain the backup and restore process
- Develop with DynamoDB by using the AWS software development kits (SDKs)

Module overview

Sections

1. Introduction
2. Introducing AWS database options
3. Key concepts for DynamoDB
4. Partitions and data distribution
5. Secondary indexes
6. Read/write throughput
7. Streams and global tables
8. Backup and restore
9. Basic operations for DynamoDB tables

Lab

Working with Amazon DynamoDB

Activity

Calculating RCUs and WCUs



Knowledge check



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

4

This module includes the following sections:

1. Introducing the café scenario
2. Introducing AWS database options
3. Key concepts for DynamoDB
4. Partitions and data distribution
5. Secondary indexes
6. Read/write throughput
7. Streams and global tables
8. Backup and restore
9. Basic operations for DynamoDB tables

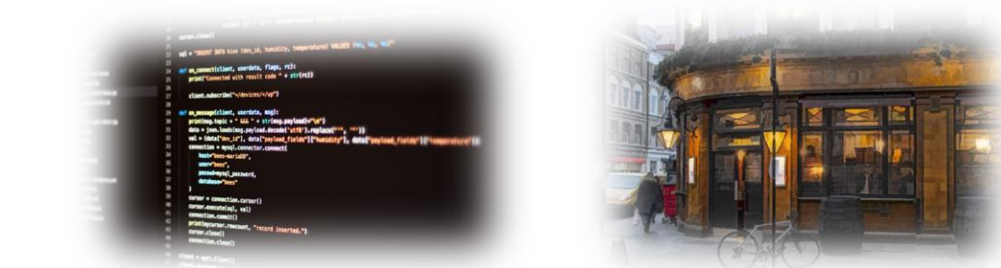
This module also includes:

- An activity where you will calculate read capacity units (RCUs) and write capacity units (WCUs)
- A lab about working with Amazon DynamoDB

Finally, you will complete a knowledge check to test your understanding of key concepts covered in this module.

Café business requirement

The café has a basic website now. Sofia wants to modify the site so that information can be updated dynamically and staff members can retrieve information about the product inventory.

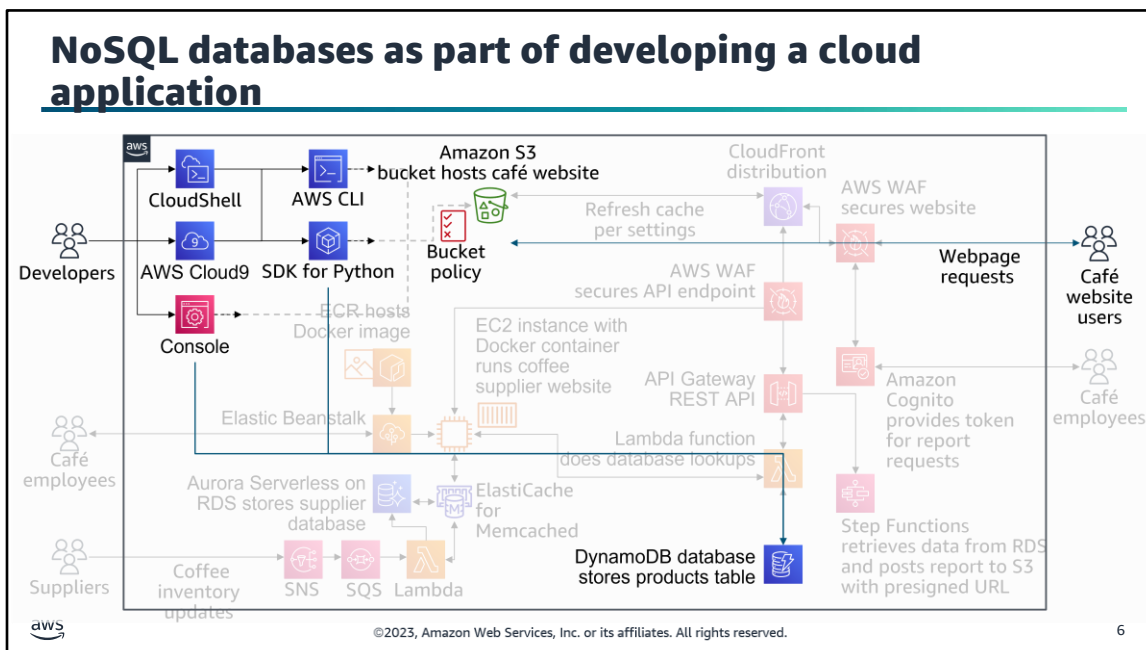


©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

5

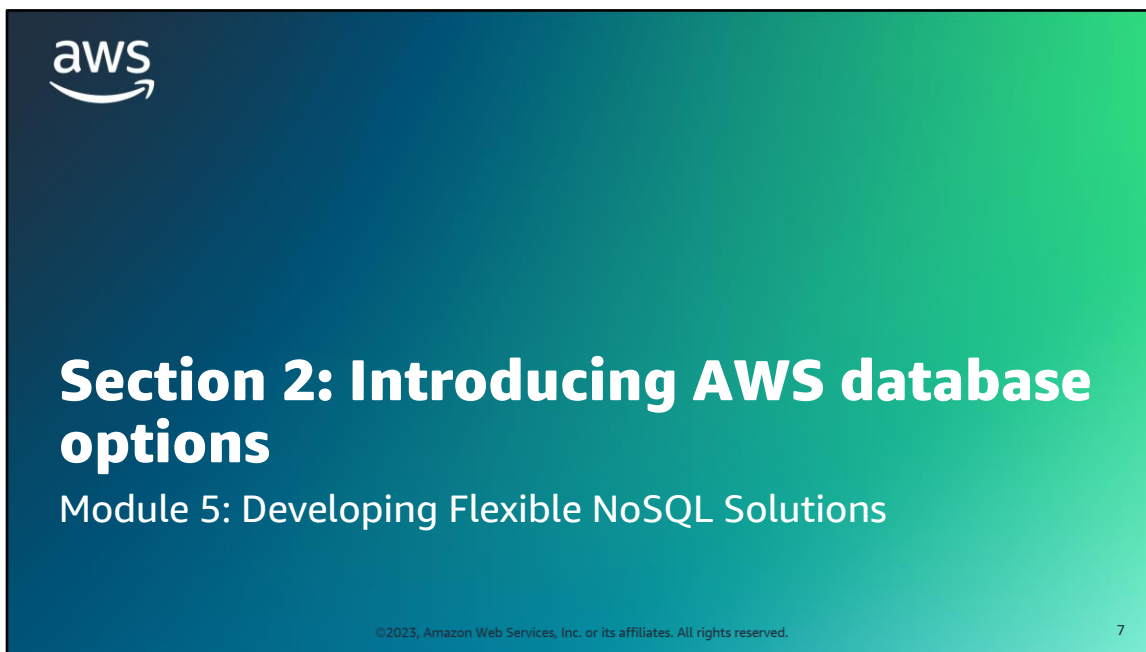
The café has a basic website now. Sofia wants to modify the site so that information can be updated dynamically and staff members can retrieve information about the product inventory.

Sofia decides to store this information in a DynamoDB database. As a proof of concept, she decides to create a script that retrieves all of the inventory items from the table and another script that uses a product name to retrieve a single record.



The diagram on this slide gives an overview of the application that you will build through the labs in this course.

The highlighted portions are relevant to this module. As highlighted in the diagram, you will set up a DynamoDB database to store the product information.



Section 2: Introducing AWS database options.

Database types

Databases (DBs) are excellent solutions for persistent storage.

The two major types of databases are:

- Relational databases
- Nonrelational databases

Relational

Traditional examples:

Microsoft SQL Server
Oracle Database
MySQL

Nonrelational

Traditional examples:

MongoDB
Cassandra
Redis



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

8

The two major types of databases are:

- Relational databases
- Nonrelational databases

This section defines these two types of databases, and covers AWS examples of each type.

Database type: Relational

When to choose a relational database:

- You must use strict schema rules and must enforce data quality
- Your database doesn't need extreme read/write capacity
- If you have a relational dataset that doesn't require extreme performance, a relational database management solution (RDBMS) can be the lowest-effort solution.

Relational

Traditional examples:

Microsoft SQL Server
Oracle Database
MySQL



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

9

A relational database is a persistent store where data has relationships to other data. Because of the relationships, you must enforce the schema (or the definitions of the data that's stored in the table) and you must also enforce how tables relate to one another. Changing the schema to accommodate new requirements can sometimes be a challenge.

Relational databases can provide fast performance. However, for extreme performance (latency in the millisecond or microsecond range), other databases might be a better fit, such as a nonrelational database.

Database type: Nonrelational

When to choose a nonrelational database:

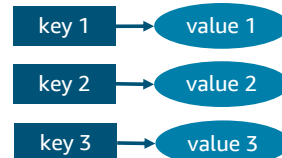
- You need your database to scale horizontally
- Your data doesn't lend itself well to traditional schemas (nonrelational databases can use key-value pairs)
- Your read/write rates exceed rates that can be economically supported through traditional structured query language (SQL) databases

Nonrelational

Traditional examples:

MongoDB
Cassandra
Redis

Key-value



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

10

Nonrelational databases—often called NoSQL databases—are schemaless stores, where items in the database are not required to conform to rules. It is not uncommon to see rows in a nonrelational database table that have different attributes.

The design of nonrelational databases means that they can scale by adding more compute to the infrastructure. That is, servers can be brought online to increase capacity when customer demand increases. When demand drops, the additional servers can be shut down to reduce costs.

In addition, nonrelational databases can deliver performance, which—for the cost—exceeds the performance that traditional structured query language (SQL) databases can support.

Comparing structured data storage

Feature	Relational or SQL	Nonrelational or NoSQL
Data Storage	Rows and columns	Key-value, documents, and graphs
Schemas	Fixed	Dynamic
Querying	SQL-based queries	Focused on collection of documents
Scalability	Vertical	Horizontal



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

11

This table compares the features of relational (or SQL) databases and nonrelational (or NoSQL) databases.

AWS database options: Relational



Amazon Relational
Database Service
(Amazon RDS)



Amazon Aurora

AWS offers more databases—these services are commonly used examples.



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

12

Amazon Relational Database Service (Amazon RDS) has different optimizations that you can use, such as memory, performance, and I/O. It offers six database engines:

- Amazon Aurora: Database that's built for the cloud and is compatible with MySQL and PostgreSQL
- PostgreSQL: Can be used to deploy a scalable PostgreSQL database with resizable hardware capacity
- MySQL: Can be used to set up a MySQL deployment
- MariaDB: Scalable MariaDB database engine
- Oracle Database: Fully managed commercial database that can be used to set up, operate, and deploy an Oracle DB in the cloud
- Microsoft SQL Server: Can be used to set up, operate, and scale Microsoft SQL Server (2012, 2014, 2016, 2017, 2021)

Amazon Aurora is a database that is built for the cloud and is compatible with MySQL and PostgreSQL. In terms of performance, Aurora can be up to five times faster than standard MySQL. It also can be up to three times faster than standard PostgreSQL databases. It is a fully managed service. It provides security, availability, and reliability at about one tenth of the cost of commercial databases.

AWS database options: Nonrelational



Amazon DynamoDB



Amazon ElastiCache



Amazon Neptune

AWS offers more databases—these services are commonly used examples.



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

13

Amazon DynamoDB is a key-value and document database. It's a multi-Region, multi-active, and durable database that delivers single-digit millisecond performance. It works well for internet-scale applications. It supports atomicity, consistency, isolation, and durability (ACID) transactions. Thus, critical business applications can use DynamoDB for persistent storage.

Amazon ElastiCache is an in-memory data store that can boost the performance of databases. ElastiCache can improve the performance of caching, session stores, gaming, geospatial services, real-time analytics, and queueing.

Amazon Neptune is a graph database engine that's optimized for storing billions of relationships. Its queries run with millisecond latency. Developers can use Neptune for recommendation engines, fraud detection, drug discovery, network security, and other solutions that need complex relationships to find solutions.

Section 2 key takeaways



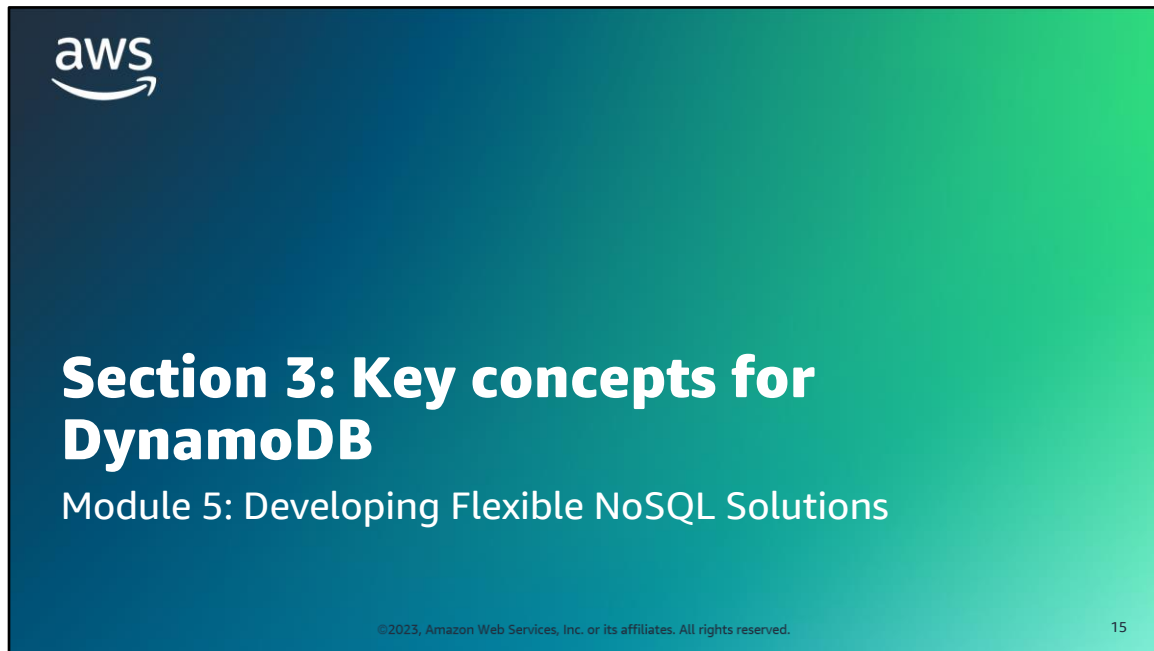
- Relational databases:
 - Strict schema
 - Doesn't need extreme performance
- Nonrelational databases:
 - Horizontal scaling
 - Non-traditional schema
- AWS offers several different services to support both relational and nonrelational databases.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

14

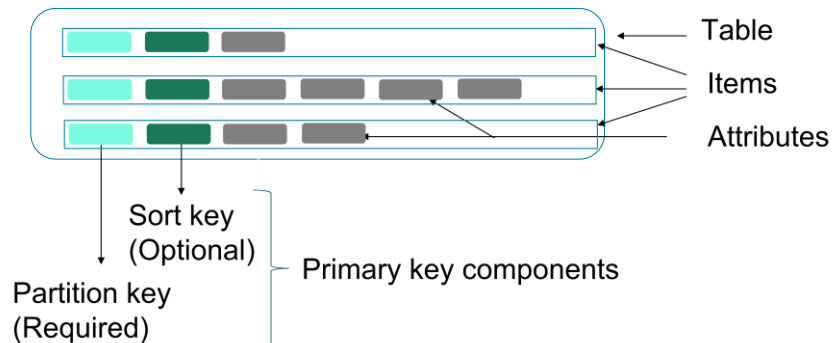
The following are the key takeaways from this section of the module:

- Relational databases:
 - Strict schema
 - Doesn't need extreme performance
- Nonrelational databases:
 - Horizontal scaling
 - Non-traditional schema
- AWS offers several different services to support both relational and nonrelational databases.



Section 3: Key concepts for DynamoDB.

Basic components of DynamoDB



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

16

The basic components of DynamoDB are:

- **Tables:** DynamoDB stores data in tables. A table contains items with attributes.
- **Items:** Each table contains zero or more items. An item is a group of attributes that is uniquely identifiable among all the other items.
- **Attributes:** Each item is composed of one or more attributes. An attribute is a fundamental data element, something that doesn't need to be broken down any further.
- **Primary key:** A table has a primary key that uniquely identifies each item in the table. No two items can have the same key.

When these components are compared to the components of a relational database table, items are analogous to rows and attributes are analogous to columns.

Types of primary keys

Simple primary key
(partition key only)

SensorId (Partition Key)	Latitude	Longitude
SensorA	40.712784	-74.005941
SensorB	35.689488	139.691706

SensorLocation table

Composite primary key
(partition key and sort key)

SensorId (Partition Key)	Time (Sort Key)	Value
SensorA	2021-01-03T10:15:30	30
SensorA	2021-01-04T10:19:30	35
SensorB	2021-03-04T11:21:20	28

SensorReadings table



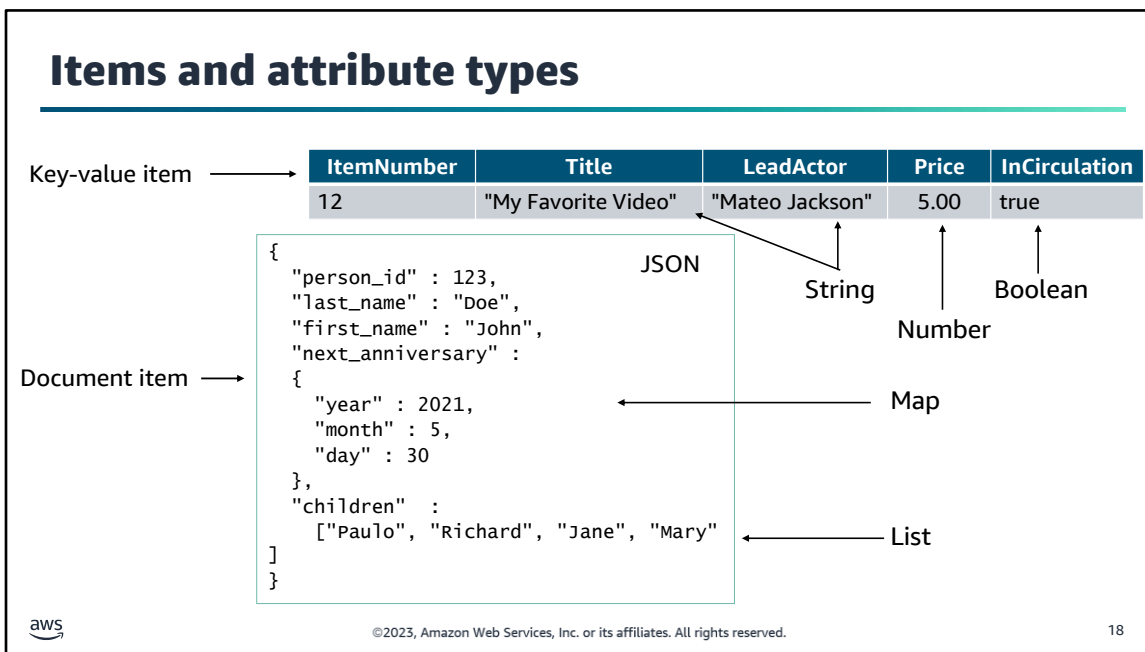
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

17

DynamoDB supports two types of primary keys:

- **Simple primary key:** The simple primary key is composed of one attribute that's known as the *partition key*. DynamoDB builds an unordered index on this primary key attribute. Each item in the table is uniquely identified by its partition key value.
 - In the SensorLocation table, the primary key is a *partition key* because it consists of a single attribute, *SensorId*, which is the partition key. Each item in the table is uniquely identified by its partition key value (*SensorA* and *SensorB*). Also, each sensor has exactly one location, which is expressed in latitude and longitude.
- **Composite primary key:** The composite primary key is composed of two attributes: the *partition key* and the *sort key*. DynamoDB builds an unordered index on the partition key attribute and a sorted index on the sort key attribute. In a table that has a partition key and a sort key, it's possible for two items to have the same partition key value. However, those two items must have different sort key values.
 - In the SensorReadings table, the primary key is a *partition and sort primary key*. It is composed of the *SensorId* attribute (the partition key) and the *Time* attribute (the sort key). Each *SensorId* might have multiple items that correspond to sensor readings at different times. The combination of *SensorId* and *Time* uniquely identifies items in the table. With this design, you can query the table for all readings that are related to a particular sensor.

Remember that DynamoDB can be used as a key-value store and document store. In the examples shown, the primary key value (partition key and sort key if present) is the key. The remaining attributes constitute the value that corresponds to the key.



Unlike a relational database, DynamoDB isn't constrained by a predefined schema. An item can have any number of attributes with different value types. Each attribute has a name and a value. An attribute value can be one of the following types:

- Scalar types: Number, string, binary, Boolean, and null
- Set types (multi-value types): String set, number set, and binary set
- Document types: List and map

The size of an item is the sum of the lengths of its attribute names and values. An item can be up to 400 KB in size.

Section 3 key takeaways



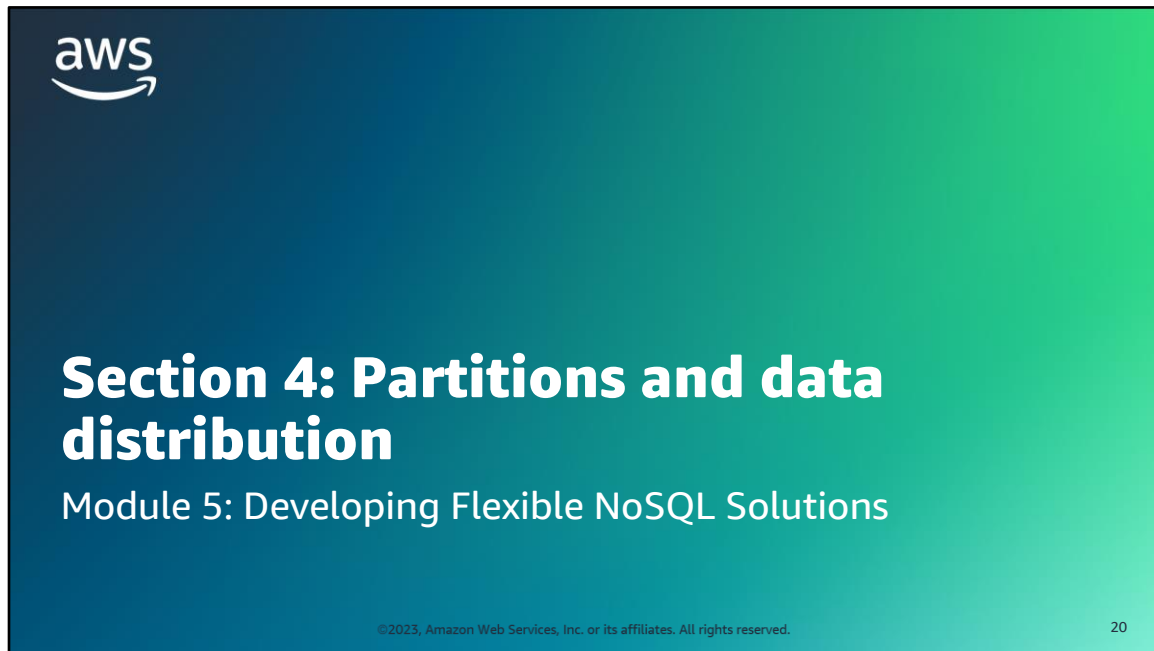
- Components of DynamoDB:
 - Table
 - Items
 - Attributes
- Types of primary keys:
 - Simple primary key
 - Composite primary key
- An item has attributes.
- Different items in the same table can have a different number of attributes.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

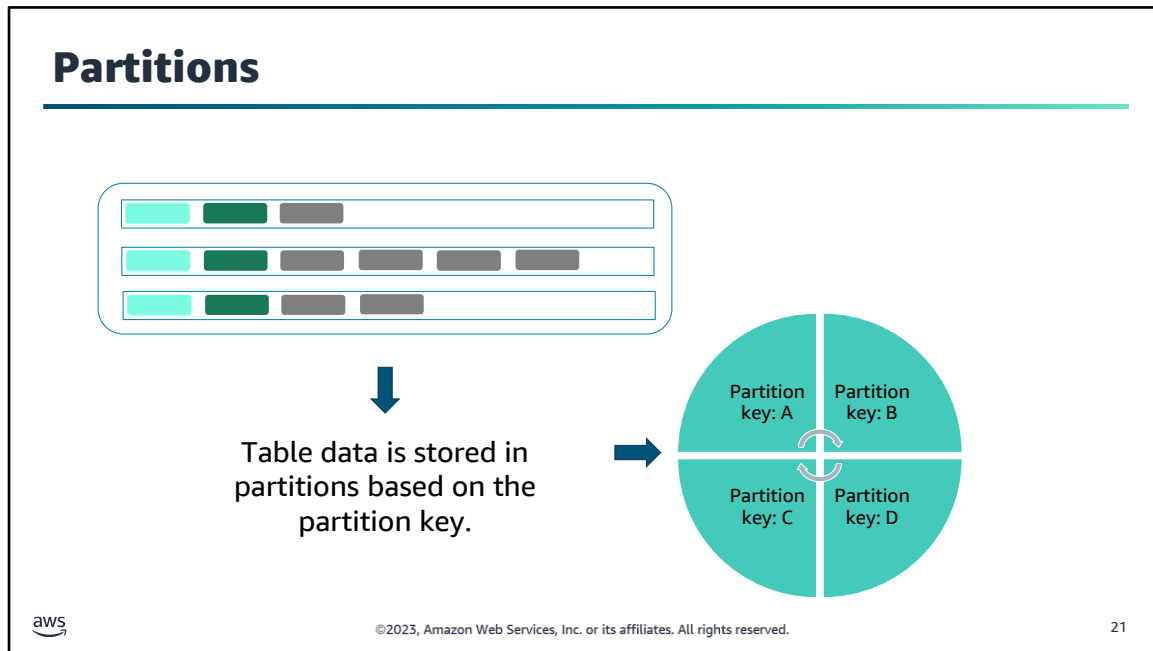
19

The following are the key takeaways from this section of the module:

- Components of DynamoDB:
 - Table
 - Items
 - Attributes
- Types of primary keys:
 - Simple primary key
 - Composite primary key
- An item has attributes.
- Different items in the same table can have a different number of attributes.



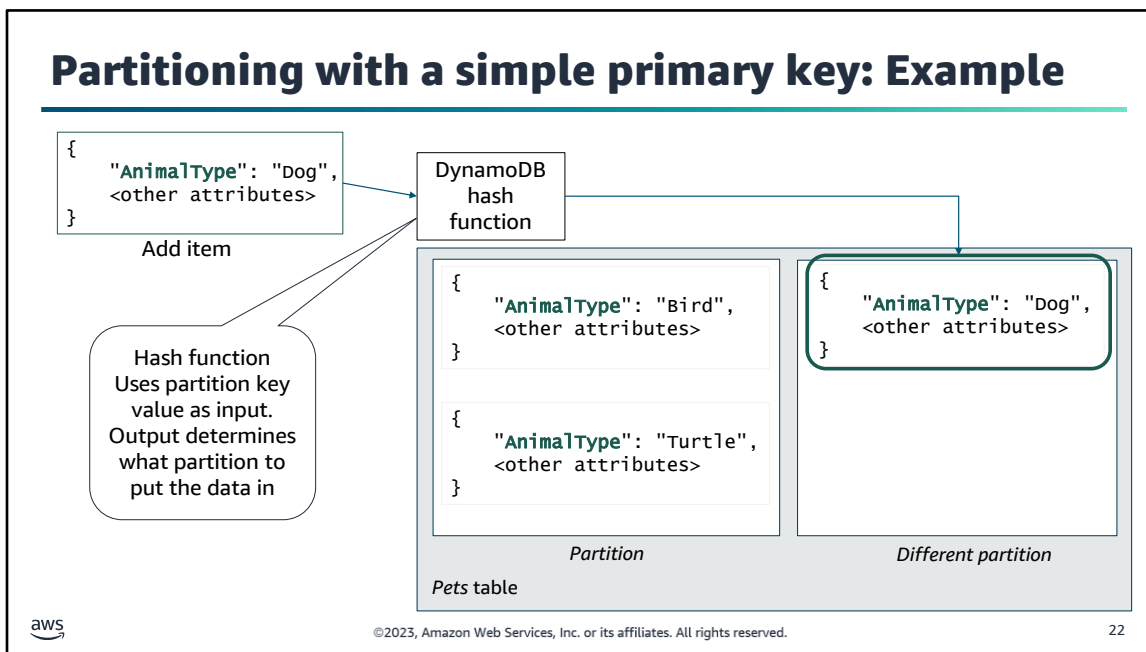
Section 4: Partitions and data distribution.



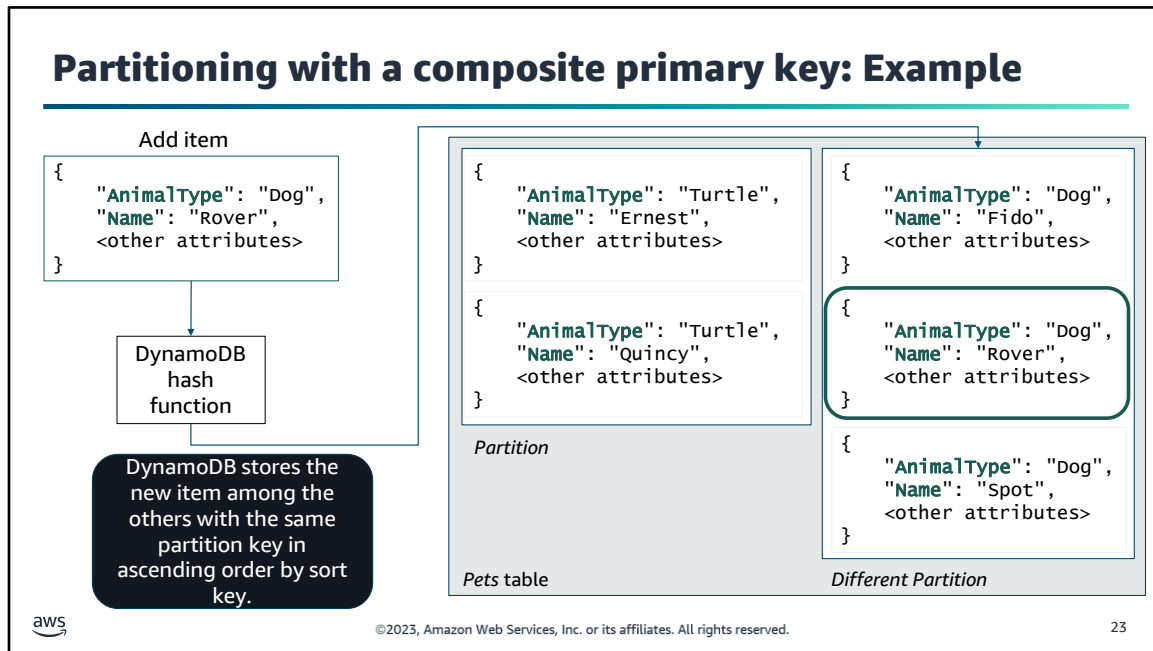
DynamoDB stores data in partitions. A *partition* is an allocation of storage for a table. It's backed by solid state drives (SSDs), and is automatically replicated across multiple Availability Zones within an AWS Region. Partition management is handled entirely by DynamoDB.

If your table has a simple primary key (partition key only), DynamoDB stores and retrieves each item based on its partition key value. The partition key of an item is also known as its *hash attribute*. A table can have a composite primary key (partition key and sort key). In this case, DynamoDB will store all the items that have the same partition key value physically close together. It will then order them by sort-key value in the partition. The sort key of an item is also known as its *range attribute*.

For information about how partitioning works, see “Partitions and Data Distribution” at <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.Partitions.html>.



In this example, the *Pets* table has a simple primary key (partition key only) and spans multiple partitions. DynamoDB calculates the hash value of the partition key to determine which partition should contain the item. In this case, the partition is based on the hash value of the string *Dog*. Notice that the items are not stored in sorted order.



Suppose that the *Pets* table has a composite primary key that consists of *AnimalType* (partition key) and *Name* (sort key). DynamoDB again calculates the hash value of the partition key to determine which partition should contain the item. In that partition, several items have the same partition key value. Therefore, DynamoDB stores the new item among the others with the same partition key, in ascending order by sort key. In this example, DynamoDB writes an item with a partition key value of *Dog* and a sort key value of *Rover* in ascending order.

Section 4 key takeaways



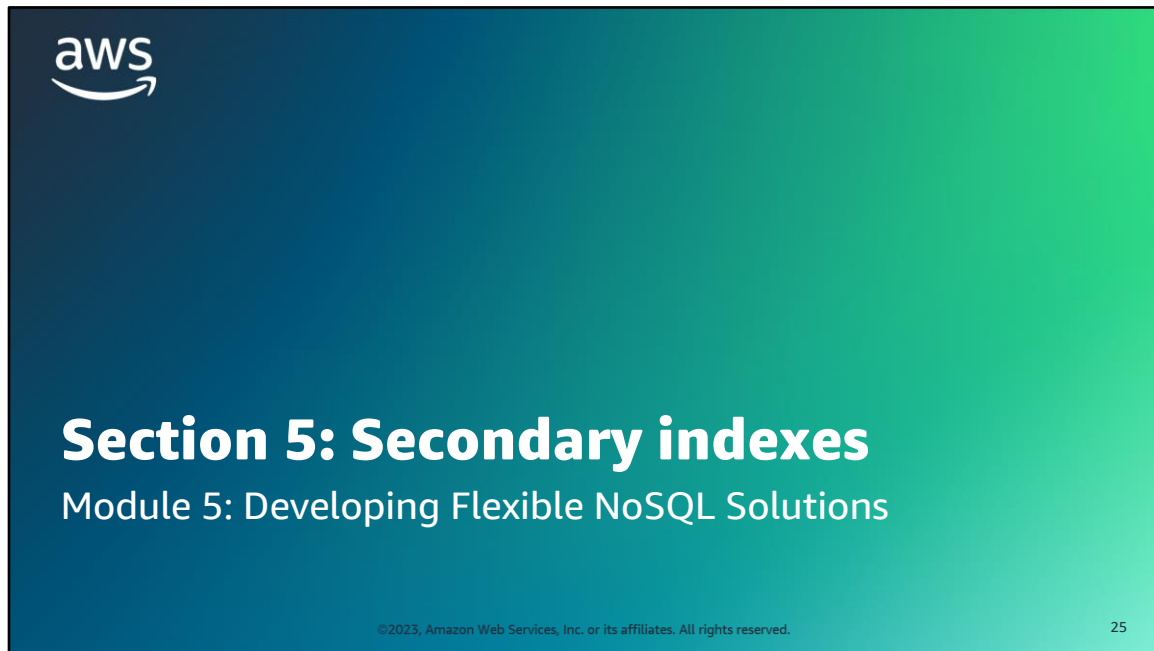
- DynamoDB stores data in partitions based on the partition key.
- With a simple primary key, the key determines which partitions are selected.
- With a composite primary key, the key still determines which partitions are selected on the key, but the order is based on the sort key.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

24

The following are the key takeaways from this section of the module:

- Amazon DynamoDB stores data in partitions based on the partition key.
- With a simple primary key, the key determines which partitions are selected.
- With a composite primary key, the key still determines which partition is selected, but the order is based on the sort key.



Section 5: Secondary indexes.

Secondary indexes

- Use to query data based on non-primary-key attributes
 - Secondary index defines an alternate key
- Contain:
 - Alternate-key attributes
 - Primary-key attributes
 - Optional subset of other attributes from the base table (projected attributes)
- Can be one of two types:
 - Global secondary index
 - Local secondary index



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

26

You can use a *secondary index* to perform queries on attributes that aren't part of the table's primary key. With a secondary index, you can query the data in the table by using an alternate key, in addition to queries against the primary key.

In addition to the alternate key attributes and primary key attributes (partition key and sort key), a secondary index contains a subset of the other table attributes. When you create an index, you specify which attributes will be copied (or projected) from the base table to the index. At a minimum, DynamoDB projects the key attributes from the base table into the index.

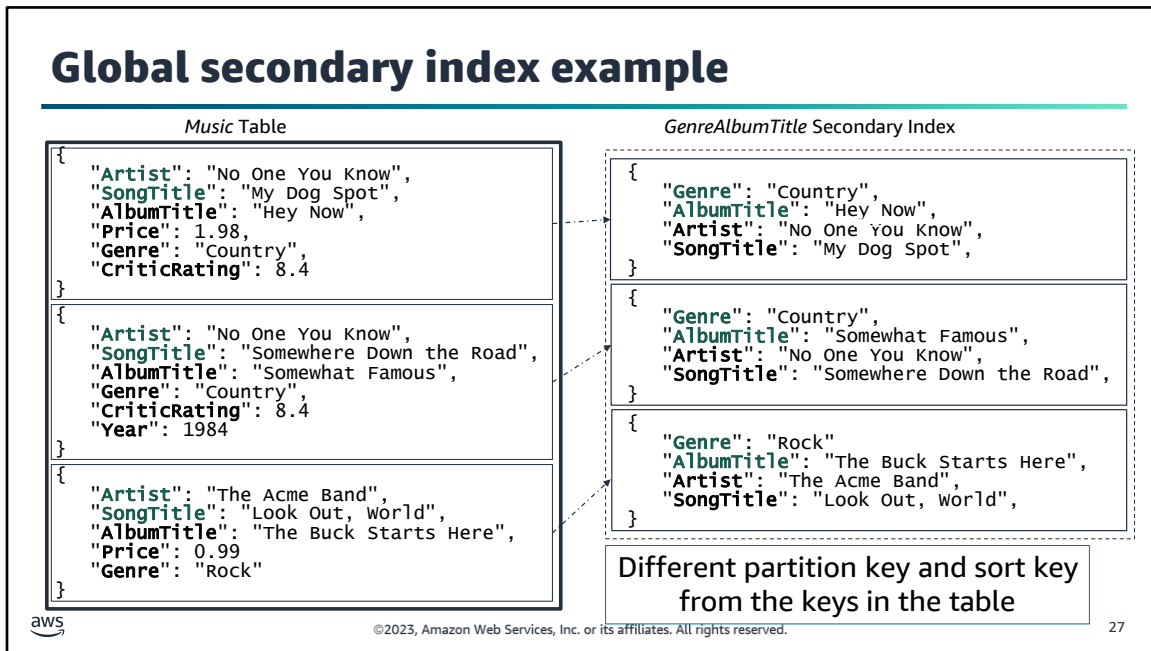
DynamoDB supports two types of secondary indexes:

- *Global secondary index*: An index with a partition key and a sort key, both of which can be different from the keys in the base table. A global secondary index is considered *global* because queries on the index can span all the data in the base table, across all partitions. A global secondary index has no size limitations. It also has its own provisioned throughput settings for read/write activity that are separate from the throughput settings of the table.
- *Local secondary index*: An index that has the same partition key as the base table, but a different sort key. A local secondary index is *local* because every partition is scoped to a base table partition that has the same partition key value. As a result, the total size of indexed items for any one partition key value can't exceed 10 GB. Also, a local secondary index shares provisioned throughput settings for read/write activity with the table that it indexes.

Each table in DynamoDB can have up to 20 global secondary indexes (default limit) per table, and up to five local secondary indexes per table.

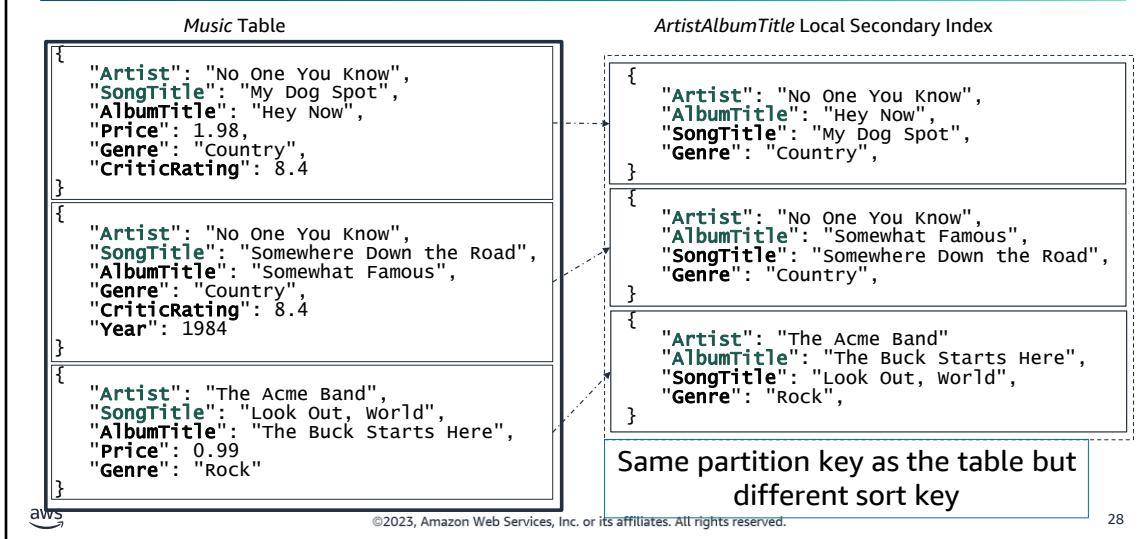
For more information about secondary indexes, see “Improving Data Access with Secondary Indexes” at

<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/SecondaryIndexes.html>.



The slide shows an example of when you would use a global secondary index. Suppose that you have a *Music* table. You can query data items by *Artist* (partition key) or by *Artist* and *SongTitle* (partition key and sort key). What if you also wanted to query the data by *Genre* and *AlbumTitle*? You cannot do this query with the *Music* table. To do this query, you create a global secondary index on *Genre* and *AlbumTitle* from the base *Music* table. Then, you query the index in much the same way that you would query the *Music* table. In the index, *Genre* is the partition key and *AlbumTitle* is the sort key. Notice that the combination of *Genre* and *Album Title* might not be unique. Multiple albums can belong to one genre.

Local secondary index example



Suppose that you want to query by *Artist* and by *AlbumTitle*. In this case, you would create a local secondary index (called *ArtistAlbumTitle*) from the base *Music* table. The local secondary index has the same partition key as the base table (*Artist*) but a different sort key (*AlbumTitle*).

For a local secondary index, the partition key is the same as the table's partition key. The sort key can be any scalar attribute.

Section 5 key takeaways



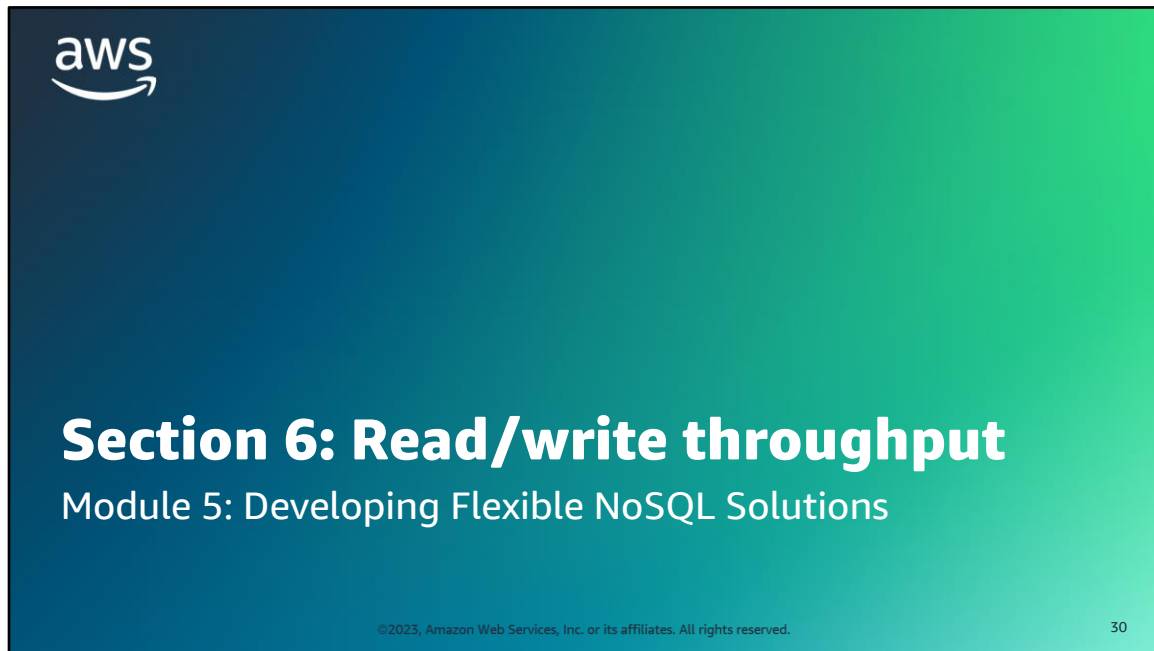
- Secondary indexes contain a subset of attributes and an alternate key or keys to support query operations.
- Global secondary indexes have different partition keys and sort keys from the keys in the base table.
- Local secondary indexes have the same partition key as the base table, but a different sort key.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

29

The following are the key takeaways from this section of the module:

- Secondary indexes contains a subset of attributes and an alternate key or keys to support query operations.
- Global secondary indexes have different partition keys and sort keys from the keys in the base table.
- Local secondary indexes have the same partition key as the base table, but a different sort key.



Section 6: Read/write throughput.

Read consistency

- DynamoDB maintains multiple copies of data for durability.
 - All copies of your data are usually consistent within a second after a write operation.
- Eventually consistent read might return slightly stale data if a read operation is performed immediately after a write operation.
 - Use case: Blog posts.
- Strongly consistent read returns the most up-to-date data.
 - Use cases: Scoreboards, booking systems, financial algorithms.
- You can specify the preferred consistency level for reading data.



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

31

DynamoDB automatically replicates your data across multiple Availability Zones in an AWS Region, which provides built-in high availability and data durability. All copies of your data are usually consistent within a second after a write operation.

DynamoDB supports *eventually consistent* and *strongly consistent* reads.

- *Eventually consistent reads*: When you read data from a DynamoDB table, the response might not reflect the results of a recently completed write operation. The response might include some stale data. If you repeat your read request after a short time, the response should return the latest data.
- *Strongly consistent reads*: When you request a strongly consistent read, DynamoDB returns a response with the most up-to-date data. The result reflects the updates from all prior write operations that were successful. A strongly consistent read might not be available if a network delay or outage occurs. Consistent reads are not supported on global secondary indexes.

You can specify the desired consistency level for reading data.

For more information, see “Read Consistency” at

<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.ReadConsistency.html>.

Transactions

- Simplify the developer experience of making coordinated, all-or-nothing changes (inserts, deletes, or updates) to multiple items both within and across tables
- Provide atomicity, consistency, isolation, and durability (ACID), which help you maintain data correctness in your applications
- Use cases
 - Processing financial transactions
 - Fulfilling and managing orders
 - Building multiplayer game engines
 - Coordinating actions across distributed components and services



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

32

Amazon DynamoDB transactions simplify the developer experience of making coordinated, all-or-nothing changes to multiple items both within and across tables. Transactions provide atomicity, consistency, isolation, and durability (ACID) in DynamoDB. ACID transactions help you maintain data correctness in your applications.

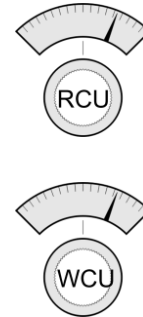
Many use cases can be implemented by using transactions, for example:

- Processing financial transactions
- Fulfilling and managing orders
- Building multiplayer game engines
- Coordinating actions across distributed components and services

For more information about Amazon DynamoDB transactions, see “Managing Complex Workflows with DynamoDB Transactions” at <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/transactions.html>.

Provisioned read/write throughput

- Throughput = Maximum amount of capacity that an application can consume from a table or index
- It is divided evenly among partitions
- Read capacity unit (RCU)
 - Number of strongly consistent reads per second of items that are up to 4 KB in size
 - Eventually consistent reads use half of the provisioned read capacity
- Write capacity unit (WCU)
 - Number of 1-KB writes per second



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

33

Amazon DynamoDB supports both provisioned and on-demand throughput. Provisioned throughput is the maximum amount of capacity that an application can consume from a table or index. If your application exceeds your provisioned throughput capacity on a table or index, it's subject to request throttling.

DynamoDB divides throughput evenly among partitions. The throughput per partition is the total provisioned throughput divided by the number of partitions.

With provisioned throughput, you specify the throughput capacity in terms of read capacity units (RCUs) and write capacity units (WCUs):

- A read capacity unit is the number of strongly consistent reads per second of items that are up to 4 KB in size. If you perform eventually consistent reads, you use half of the read capacity units that are provisioned. In other words, for eventually consistent reads, one read capacity unit is two reads per second for items that are up to 4 KB.
- A write capacity unit is the number of 1 KB writes per second.

On-demand read/write throughput

- Pay-per-request pricing for read and write requests
- DynamoDB adapts rapidly to accommodate the workload
- Request rate is limited by throughput default table limits
- Create or update a table to use on-demand mode
- Use cases
 - You create new tables with unknown workloads
 - You have unpredictable application traffic
 - You prefer the ease of paying for only what you use



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

34

When you choose *Amazon DynamoDB on-demand*, DynamoDB instantly accommodates your workloads as they scale up or scale down to any previously reached traffic level. If a workload's traffic level reaches a new peak, DynamoDB adapts to accommodate the workload. Tables that use on-demand mode deliver the same single-digit millisecond latency, service-level agreement (SLA) commitment, and security that DynamoDB already offers. You can choose on-demand mode for both new and existing tables, and you can continue using the existing DynamoDB application programming interfaces (APIs) without changing code. DynamoDB on-demand offers pay-per-request pricing for read and write requests so that you pay only for what you use.

On-demand mode is a good option when any of the following situations are true:

- You create new tables with unknown workloads.
- You have unpredictable application traffic.
- You prefer the ease of paying for only what you use.

For more information about Amazon DynamoDB on-demand and provisioned modes, see "Read/Write Capacity Mode" at

<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.ReadWriteCapacityMode.html>.

Activity: Calculating RCUs and WCUs



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

35

In this educator-led activity, you will calculate the number of read capacity units and write capacity units that are needed for a particular scenario.

Activity: Calculating RCU

- Suppose that you must read 20 items that are 11 KB in size every second, with eventual consistency.
- How many RCUs must you provision?
- 1 RCU = 1 strongly consistent, or 2 eventually consistent, reads per second for items up to 4 KB in size



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

36

Suppose that you must read 20 items that are 11 KB in size every second, with eventual consistency. How many RCUs do you need?

Calculating RCU: Answer

Answer: 30 RCUs

- Round 11 KB up to the next multiple of 4: *12*
- Divide by 4 KB per RCU: *3*
- Multiply by items read per second: *60*
- Divide by 2 for eventual consistency: *30*



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

37

Answer: You need 30 RCUs.

Compute as follows:

- Round 11 KB up to the next multiple of 4: *12*
- Divide by 4 KB per RCU: *3*
- Multiply by items read per second: *60*
- Divide by 2 for eventual consistency: *30*

Activity: Calculating WCU

- Suppose that you must write 120 items that are 7 KB in size every minute.
- How many WCUs must you provision?
- 1 WCU = 1 write per second, for items up to 1 KB in size



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

38

Suppose that you must write 120 items that are 7 KB in size every minute. How many WCUs do you need?

Calculating WCU: Answer

Answer: 14 WCUs

- WCUs required to write each 7-KB item: 7
- Number of items per second: 2
- Multiply WCU per item by items per second: *14*



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

39

Answer: You need 14 WCUs.

Compute as follows:

- WCUs required to write each 7 KB item: 7
- Number of items per second: 2
- Multiply WCU per item by items per second: *14*

Section 6 key takeaways



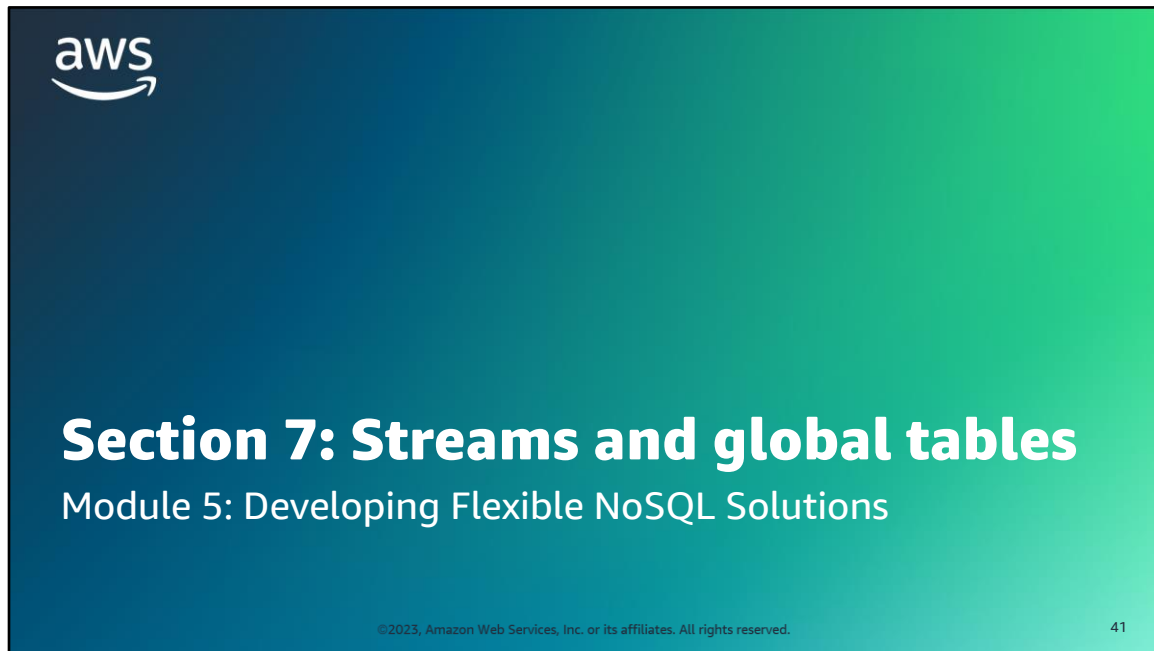
- DynamoDB provides two read consistencies:
 - Eventual consistency: Data might be stale
 - Strong consistency: Up-to-date data
- DynamoDB transactions offer insert, delete, and update operations that are all-or-nothing and ACID compliant.
- Read and write throughput can be either of the following types:
 - Provisioned (set throughput as RCU/WCU)
 - On-demand (changes with demand)

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

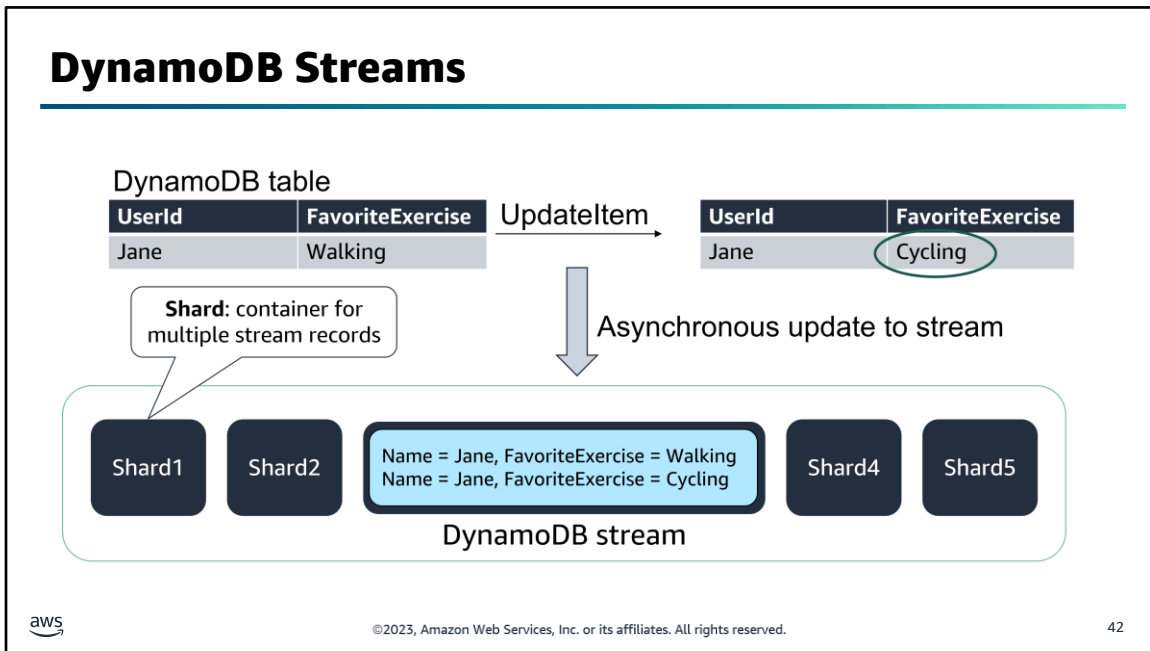
40

The following are the key takeaways from this section of the module:

- DynamoDB provides two read consistencies:
 - Eventual consistency: might be stale
 - Strong consistency: up-to-date data
- DynamoDB transactions offers insert, delete, and update operations that are all-or-nothing and ACID compliant.
- Read and write throughput can be either of the following types:
 - Provisioned (set throughput)
 - On-demand (changes with demand)



Section 7: Streams and global tables.



Applications can benefit from capturing changes to items that are stored in a DynamoDB table at the point when these changes occur. For example, consider an application that makes changes to user preferences, which are stored in a DynamoDB table. Another application, such as an ad server, must respond to the new preferences and present different advertisements.

You can enable DynamoDB Streams as a solution to this type of use case. This optional feature captures a time-ordered sequence of item-level modifications in any DynamoDB table (called a stream). It then stores this information for up to 24 hours. Applications can view the data items as they appeared both before and after they were modified, in near-real time.

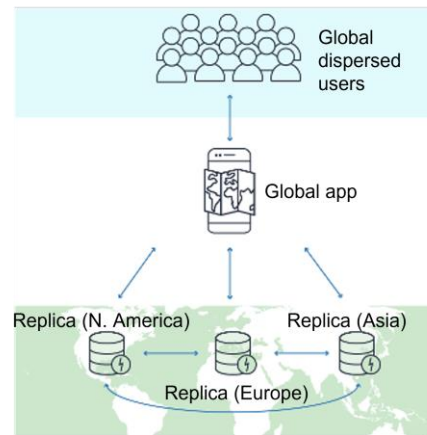
A stream consists of stream records. Each stream record represents a single data modification in the DynamoDB table that the stream belongs to. Each stream record is assigned a sequence number, which reflects the order that the record was published to the stream. When an application creates, updates, or deletes items in the table, DynamoDB Streams writes a stream record. It records the primary key attributes of the items that were modified. You can configure the stream so that the stream records capture additional information, such as the before and after images of modified items.

Stream records are organized into groups, or shards. Each shard acts as a container for multiple stream records, and it contains information that is required for accessing and iterating through these records. The stream records in a shard are automatically removed after 24 hours.

For information about working with streams, see “Change Data Capture for DynamoDB Streams” at <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Streams.html>.

Global tables

- Global tables offer a solution to deploy a multi-Region, multi-active database without needing to build and maintain your own replication solution.
- You specify the AWS Regions where you want the table to be available.



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

43

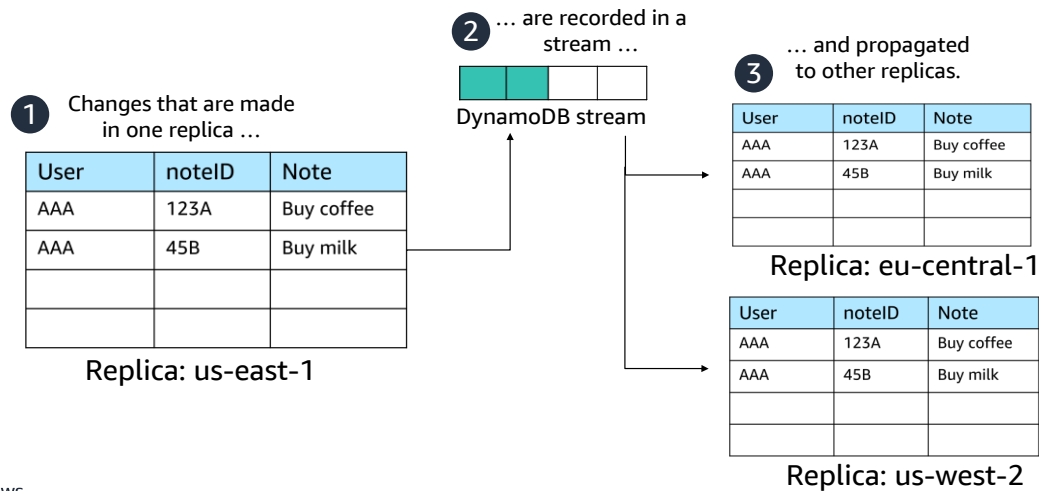
DynamoDB automatically spreads the data and traffic for your tables over a sufficient number of servers to handle your throughput and storage requirements. At the same time, it maintains consistent and fast performance. All your data is stored on solid-state disks (SSDs) and is automatically replicated across multiple Availability Zones in an AWS Region. As a result, it provides built-in high availability and data durability. You can use *global tables* to keep DynamoDB tables synchronized across AWS Regions.

Global tables provide a fully managed solution for deploying a multi-Region, multi-active database, without needing to build and maintain your own replication solution. When you create a global table, you specify the AWS Regions where you want the table to be available. DynamoDB performs all of the necessary tasks to create identical tables in these Regions, and to propagate ongoing data changes to all of them.

The following scenario illustrates one use case for a global table. Suppose that you have a large customer base that's spread across three geographic areas: North America, Europe, and Asia. Customers must update their profile information while they are using your application. To address these requirements, you could create three identical DynamoDB tables that are named *CustomerProfiles* in three different AWS Regions. These three tables would be entirely separate from each other. Changes to the data in one table would not be reflected in the other tables. Without a managed replication solution, you could write code to replicate data changes among these tables. However, this could be a time-consuming and labor-intensive effort.

Instead of writing your own code, you could create a global table that consists of your three Region-specific *CustomerProfiles* tables. DynamoDB would then automatically replicate data changes among those tables. Thus, changes to *CustomerProfiles* data in one Region would be propagated to the other Regions. In addition, if one of the AWS Regions becomes temporarily unavailable, your customers could still access the same *CustomerProfiles* data in the other Regions.

Global tables: Data replication



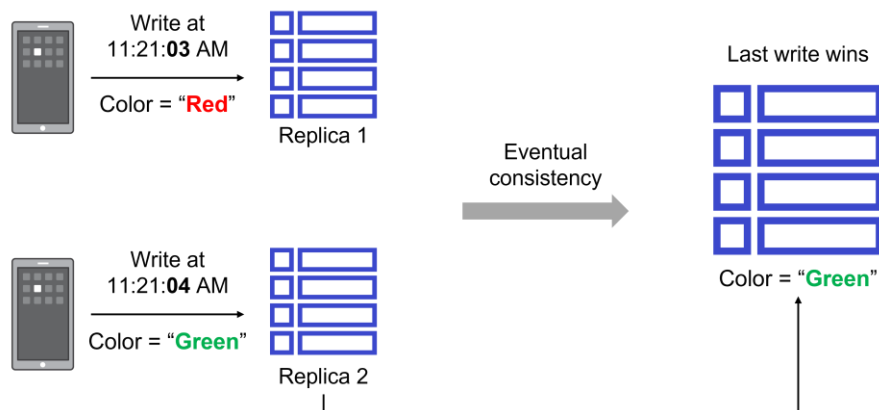
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

44

A global table is a collection of one or more DynamoDB tables, which are called *replica tables*. A replica table (or replica) is a single DynamoDB table that functions as a part of a global table. Each replica stores the same set of data items. Any given global table can only have one replica table per Region. Each replica has the same table name and the same primary key schema.

Global tables use DynamoDB Streams to propagate changes between replicas. Any changes that are made to any item in any replica table are recorded in a stream. These changes are propagated to all the other replicas in the same global table. In a global table, a newly written item is usually propagated to all replica tables within seconds. With a global table, each replica table stores the same set of data items. DynamoDB does not support partial replication of only some of the items.

Global tables: Concurrent updates

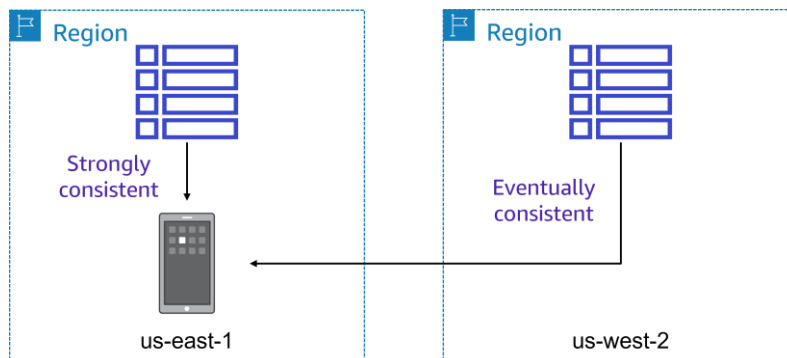


©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

45

Conflicts can arise if applications update the same item in different Regions at about the same time. To achieve eventual consistency, DynamoDB global tables use a *last writer wins* reconciliation between concurrent updates. DynamoDB makes a best effort to determine the last writer. With this conflict resolution mechanism, all the replicas will agree on the latest update. They then converge toward a state where they all have identical data.

Global tables: Read consistency



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

46

An application can read and write data to any replica table. If your application only uses eventually consistent reads, and only issues reads against one AWS Region, then it will work without any modification. However, if your application requires strongly consistent reads, then it must perform all of its strongly consistent reads and writes in the same Region. DynamoDB does not support strongly consistent reads across AWS Regions. Therefore, if you write to one Region and read from another Region, the read response might include stale data. This data doesn't reflect the results of recently completed writes in the other Region. *Strongly consistent reads require the use of a replica in the same Region where the client is running.*

Transactions are enabled for all single-Region DynamoDB tables, and they are disabled on global tables by default. You can choose to enable transactions on global tables by request, but replication across Regions is asynchronous and eventually consistent. You might observe partially completed transactions during replication to other Regions.

Section 7 key takeaways



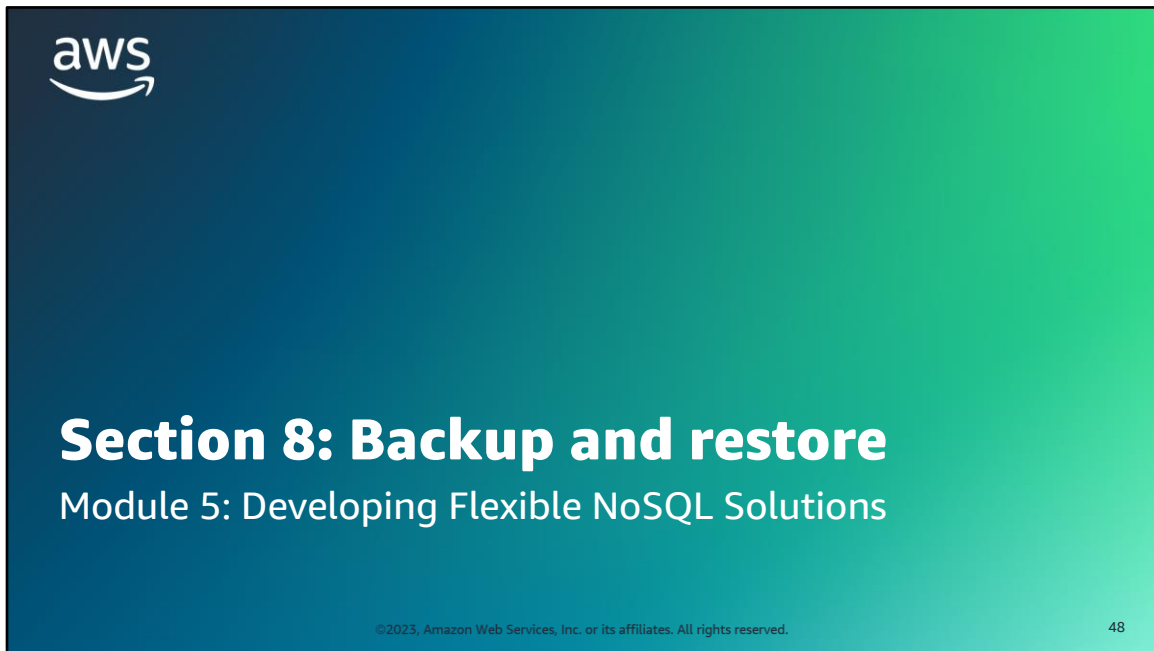
- DynamoDB streams are observed changes in data.
- Streams are composed of shards of data.
- Streams can be consumed by other AWS services, like AWS Lambda.
- Global tables replicate DynamoDB tables across AWS Regions.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

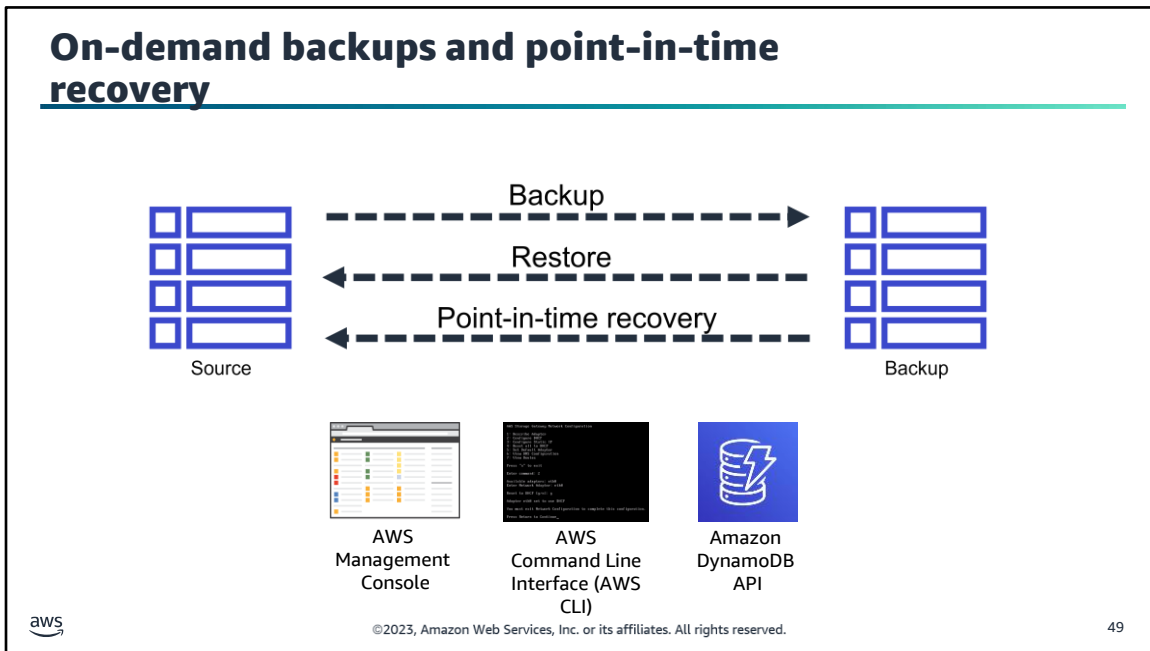
47

The following are the key takeaways from this section of the module:

- DynamoDB streams are observed changes in data.
- Streams are composed of shards of data.
- Streams can be consumed by other AWS services, like AWS Lambda.
- Global tables replicate DynamoDB tables across AWS Regions.



Section 8: Backup and restore.



On-demand backups

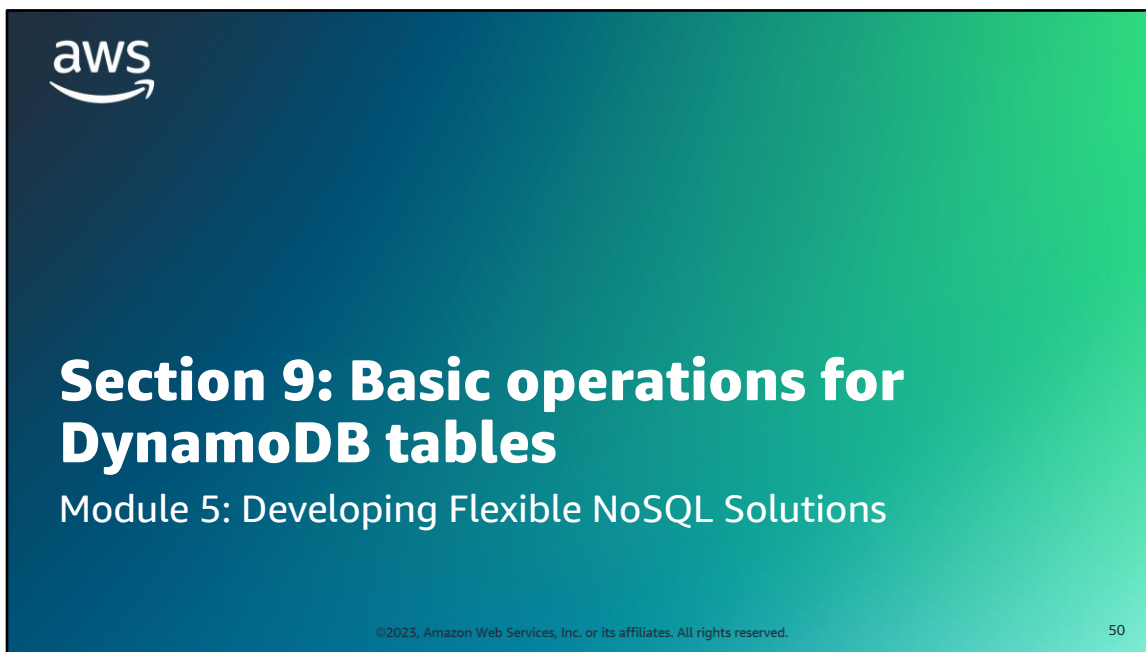
Amazon DynamoDB provides on-demand backup and restore capabilities. You can use DynamoDB to create full backups of your tables so that you can meet the requirements for long-term data retention and archiving for any data regulations that you must follow. You can back up and restore your DynamoDB table data anytime in the AWS Management Console or through an API call. Backup and restore actions run with little impact on table performance or availability.

When you create an on-demand backup, a time marker of the request is cataloged. The backup is created asynchronously by applying all changes until the time of the request to the last full table snapshot. Backup requests are processed near instantaneously, and become available for restore within minutes.

Each time you create an on-demand backup, the entire table data is backed up. No limit exists to the number of on-demand backups that you can take. All backups in DynamoDB work without consuming any provisioned throughput on the table.

Point-in-time recovery

Point-in-time recovery helps protect your Amazon DynamoDB tables from accidental write operations or delete operations. With point-in-time recovery, you don't have to worry about creating, maintaining, or scheduling on-demand backups. For example, suppose that a test script writes accidentally to a production DynamoDB table. With point-in-time recovery, you can restore that table to any point in time during the last 35 days. DynamoDB maintains incremental backups of your table.



Section 9: Basic operations for DynamoDB tables.

Amazon DynamoDB API operations

- **Control operations:** Create and manage DynamoDB tables
 - For example, CreateTable
- **Data operations:** Create, read, update, and delete actions on data in a table
 - Example: Write operations – PutItem, UpdateItem, DeleteItem,
 - Example: Read operations – GetItem, Query, Scan
- **Batch operations:** Get and write batches of items in a DynamoDB table
- **Transaction operations:** Make coordinated, all-or-nothing changes to multiple items both within and across tables



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

51

You can use the Amazon DynamoDB API to invoke the following types of operations from an application:

- *Control operations:* Use to create and manage DynamoDB tables. These operations also let you work with indexes, streams, and other objects that depend on tables.
- *Data operations:* Use to perform create, read, update, and delete actions on data in a table. These actions are the major functions that are implemented in relational database applications. Some data operations also allow you to read data from a secondary index.
- *Batch operations:* Allows developers to run multiple operations against DynamoDB. So developers can run a query, compute some value from the result of the query, and then write back to DynamoDB.
- *Transaction operations:* Make coordinated, all-or-nothing changes to multiple items both within and across tables.

For more information, see “Basic Operations on DynamoDB Tables” at <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/WorkingWithTables.Basics.html>.

Creating a table

TableName is the name of the table to create.

KeySchema specifies the attributes that make up the primary key for the table.

AttributeDefinitions is an array of attributes that describes the key schema for the table.

ProvisionedThroughput represents the provisioned throughput settings for the table.

```

table = dynamodb.create_table(
    TableName = 'users',
    KeySchema = [
        {
            'AttributeName': 'username',
            'KeyType': 'HASH'
        },
        {
            'AttributeName': 'last_name',
            'KeyType': 'RANGE'
        }
    ],
    AttributeDefinitions = [
        {
            'AttributeName': 'username',
            'AttributeType': 'S'
        },
        {
            'AttributeName': 'last_name',
            'AttributeType': 'S'
        }
    ],
    ProvisionedThroughput = {
        'ReadCapacityUnits': 5,
        'WriteCapacityUnits': 5
    }
)

```

Create_table:
Creates a new table

users Table

username (Partition key)	last_name (Sort key)
RCU 5	WCU 5

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

52

CreateTable is an asynchronous operation that creates a new table. When DynamoDB receives a *CreateTable* request, it immediately returns a response with a *TableStatus* of *CREATING*. After the table is created, DynamoDB sets the *TableStatus* to *ACTIVE*. You can perform read and write operations only on an *ACTIVE* table. You can optionally define secondary indexes on the new table as part of the *CreateTable* operation.

This example shows how to create a table by using the AWS SDK for Python (Boto3). The invocation of the *create_table* operation creates a table that is named *users*. It has a partition-and-sort primary key that's composed of attributes named *username* and *last_name* (which have key types of *hash* and *range*, respectively). The provisioned throughput for the table is five RCUs and five WCUs.

For more information about the *CreateTable* operation, see “*CreateTable*” at https://docs.aws.amazon.com/amazondynamodb/latest/APIReference/API_CreateTable.html.

Creating an item

Put_item: Creates a new item or replaces (overwrites) an existing item

Item is a map of attribute name-value pairs, one for each attribute. Only the primary key attributes are required. You can optionally provide other attribute name-value pairs for the item.

users Table: Before

username (Partition key)	last_name (Sort key)
-----------------------------	-------------------------

```
table.put_item(
    Item = {
        'username': 'janedoe',
        'first_name': 'Jane',
        'last_name': 'Doe',
        'age': 25,
        'account_type': 'standard_user'
    }
)
```

users Table:
After

username (Partition key)	last_name (Sort key)	first_name	Age	account_type
janedoe	Doe	Jane	25	standard_user



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

53

The PutItem operation creates a new item or replaces an earlier item with a new item. If an existing item in the specified table has the same primary key as the new item, the new item completely replaces the existing item. You can perform a conditional PUT operation (add a new item if an item with the specified primary key doesn't exist). Or you can replace an existing item if it has certain attribute values. You can return the item's attribute values in the same operation by using the ReturnValues parameter.

To continue with the Python example: After the *users* table is created and assigned to the *table* variable, the *put_item* operation is invoked to add a new item to the table.

For more information about the PutItem operation, see “PutItem” at https://docs.aws.amazon.com/amazondynamodb/latest/APIReference/API_PutItem.html.

Updating an item

Update_Item:
Edits an existing item's attributes, or adds a new item to the table if it doesn't already exist

users Table:
Before

username (Partition key)	last_name (Sort key)	first_name	Age	account_type
janedoe	Doe	Jane	25	standard_user

Key is the primary key of the item to be updated.

UpdateExpression defines one or more attributes to be updated.

ExpressionAttributeValues is one or more values that can be substituted in an expression.

```
table.update_item(  
    Key = {  
        'username': 'janedoe',  
        'last_name': 'Doe'  
    },  
    UpdateExpression = 'SET age = :val1',  
    ExpressionAttributeValues = {  
        ':val1': 26  
    }  
)
```

users Table:
After

username (Partition Key)	last_name (Sort key)	first_name	Age	account_type
janedoe	Doe	Jane	26	standard_user



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

54

Use the `UpdateItem` operation to update an existing item's attributes, or add a new item to the table if it doesn't already exist. You add, set, or remove attribute values. You can also perform a conditional update on an existing item. For example, you can replace an existing name-value pair if it has certain expected attribute values. Additionally, you can return the item's attribute values—from either before or after the update—by using the `ReturnValues` parameter.

In this Python example, the `update_item` operation is used to modify the `age` attribute of an item in the `users` table.

For more information about the `UpdateItem` operation, see “`UpdateItem`” at https://docs.aws.amazon.com/amazondynamodb/latest/APIReference/API_UpdateItem.html.

Deleting an item

Delete_Item:
Deletes a single
item in a table by
primary key

users Table:
Before

username (Partition key)	last_name (Sort key)	first_name	Age	account_type
janedoe	Doe	Jane	26	standard_user

Key represents the
primary key of the
item to delete.

```
table.delete_item(  
    key = {  
        'username': 'janedoe',  
        'last_name': 'Doe'  
    }  
)
```

users Table:
After

username (Partition key)	last_name (Sort key)
-----------------------------	-------------------------



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

55

Use the DeleteItem operation to delete an item in a table by using its primary key. You can perform a conditional delete operation that deletes the item if it has an expected attribute value. In addition to deleting an item, you can also return the item's attribute values by using the ReturnValues parameter.

In this Python example, the delete_item operation is used to delete an item from the **users** table.

For more information about the DeleteItem operation, see “DeleteItem” at https://docs.aws.amazon.com/amazondynamodb/latest/APIReference/API_DeleteItem.html.

Condition expressions

AccountStatus Table

userId (primary key)	lastFailedLoginTime	accountLocked
1	2021-06-06T19:20+01:00	Y

UpdateItem Operation

Primary key: userId = 1

Set: accountLocked = N

ConditionExpression:

currentLoginTime > lastFailedLoginTime + 24 hours



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

56

For the PutItem, UpdateItem, and DeleteItem operations, you can specify a condition expression to determine which items should be modified. If the condition expression evaluates to true, the operation succeeds. Otherwise, the operation fails.

This example illustrates an important concept. The accountLocked attribute in the AccountStatus table can be set to N only if the last failed login attempt was more than 24 hours earlier.

For more information about performing conditional writes, see “Condition Expressions” at

<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Expressions.ConditionExpressions.html>.

Reading an item

Get_Item:
Reads an item from a table

users
Table

username (Partition key)	last_name (Sort key)	first_name	Age	account_type
janedoe	Doe	Jane	25	standard_user

Key represents the primary key of the item to retrieve.

```

response = table.get_item(
    Key = {
        'username': 'janedoe',
        'last_name': 'Doe'
    }
)
item = response['Item']
print (item)

```

Expected output

```

{u'username': u'janedoe',
 u'first_name': u'Jane',
 u'last_name': u'Doe',
 u'account_type': u'standard_user',
 u'age': Decimal('25')}

```

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

57

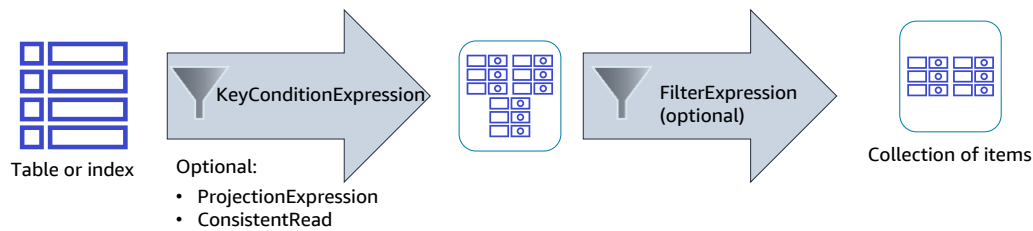
Use the `GetItem` operation to retrieve a specific item from a DynamoDB table. You must specify the table name and full primary key (partition key and sort key, if any) to retrieve a single item from a table. You can optionally specify a projection expression to retrieve only certain attributes instead of retrieving the entire item. By default, all attributes of the item are returned. You can also request that the operation use a strongly consistent read instead of the default eventually consistent read.

In this Python example, the `get_item` operation is used to retrieve an item in the `users` table, which is then printed to standard output. The expected result is a JSON object that contains all the attributes of the item.

For more information, see the following resources:

- “GetItem” at https://docs.aws.amazon.com/amazondynamodb/latest/APIReference/API_GetItem.html.
- “Working with items” at <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/WorkingWithItems.html>.

Querying a table



```
response = table.query (
    KeyConditionExpression = Key('username').eq('johndoe')
)
```



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

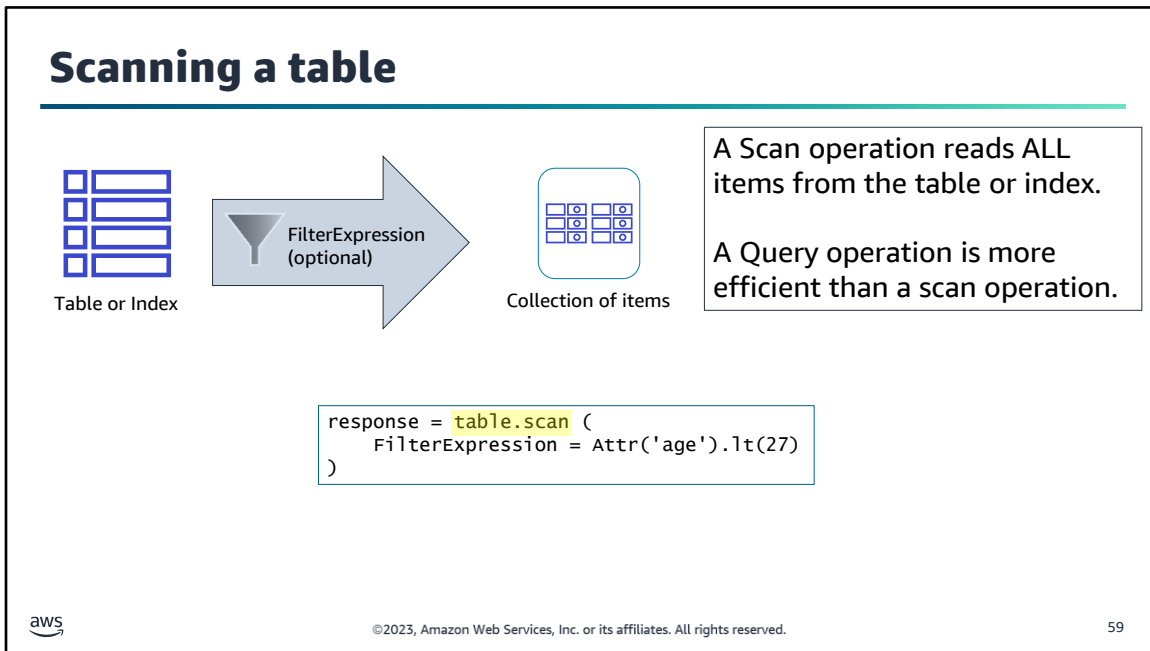
58

Use the Query operation to read only the items that match the primary key from a table or secondary index. The primary key is specified in the key condition expression. If a filter expression is specified, then the Query operation further refines the result set based on the filter. You must explicitly specify the name of the table or secondary index that you want to query. The Query operation returns a result set with the items that match the conditions that were specified. If none of the items satisfy the given criteria, then the Query operation returns an empty result set.

This Python example uses the Query operation to retrieve all the items in the users table with a primary key value that's equal to johndoe.

For more information, see the following resources:

- “Query” at https://docs.aws.amazon.com/amazondynamodb/latest/APIReference/API_Query.html.
- “Working with Queries in DynamoDB” at <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Query.html>.



The Scan operation is similar to a Query operation, but the Scan operation reads all items from the table or index. The result set can be refined by using a filter expression.

In this Python example, the Scan operation is used to retrieve all the **users** whose age is less than 27.

Because the Scan operation reads all items from the table or index, it's an expensive operation. It's more efficient to perform a Query operation that has the appropriate key condition expression to return only the data that your application needs. If you must do a Scan operation, perform a parallel scan when possible.

For more information on scan, see:

- "Scan" at https://docs.aws.amazon.com/amazondynamodb/latest/APIReference/API_Scan.html.
- "Working with Scans in DynamoDB" at <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Scan.html>.
- "Parallel Scan" at <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Scan.html>.

Optimizing performance and cost: Limiting returned data

- The 1-MB pagination limit reduces the amount of data that Query or Scan operations return.
- The Limit parameter specifies the maximum number of items that a Query or Scan operation returns.



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

60

To improve performance and optimize cost, you can limit the amount of data that Query and Scan operations return. Two factors affect the number of items that a Query or Scan operation returns:

- **Pagination limit:** By default, DynamoDB divides the results of Query and Scan operations into *pages* of data that are 1 MB in size (or less). An application can process the first page of results, then the second page, and so on. A single Query or Scan will only return a result set that fits within the 1-MB size limit.
- **Limit parameter value:** You can also limit the number of items that Query and Scan operations return in the result. To achieve this result, set the Limit parameter to the maximum number of items that you want.

For more information on optimizing performance, see:

- “Paginating Table Query Results” at <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Query.Pagination.html>.
- “Limiting the Number of Items in the Result Set” at <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Query.html#Query.Limit>.
- “Best Practices for Querying and Scanning Data” at <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/bp-query-scan.html>.

Batch operations

- **BatchGetItem:** Read up to 16 MB of data that consists of up to 100 items from multiple tables.
- **BatchWriteItem:** Write up to 16 MB of data that consists of up to 25 PUT or DELETE requests to multiple tables.
- If one request in a batch fails, the entire operation does not fail.
 - Retry with failed keys and data that is returned by the operation.



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

61

You can use batch operations to achieve higher throughput by writing, deleting, or replacing multiple items in a single request. You can also use batch operations to take advantage of parallelism without needing to manage multiple threads on your own. You can achieve lower average latencies, compared to using single PutItem or DeleteItem operations, when multiple items are written, deleted, or replaced.

You can use the following operations to retrieve or write data to DynamoDB in batches:

- **BatchGetItem:** Returns the attributes of one or more items from one or more tables. You identify requested items by primary key. A single operation can retrieve up to 16 MB of data, which can contain as many as 100 items.
- **BatchWriteItem:** Puts or deletes multiple items in one or more tables. A single call to BatchWriteItem can write up to 16 MB of data, which can comprise as many as 25 PUT or DELETE requests. Individual items to be written can be as large as 400 KB.

With batch operations, if one request in a batch fails, the entire operation does not fail.

For more information on batch operations, see:

- “Batch Operations” at <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/WorkingWithItems.html#WorkingWithItems.BatchOperations>.
- “BatchGetItem” at https://docs.aws.amazon.com/amazondynamodb/latest/APIReference/API_BatchGetItem.html.
- “BatchWriteItem” at https://docs.aws.amazon.com/amazondynamodb/latest/APIReference/API_BatchWriteItem.html.

Transactional

- **TransactWriteItems**

- Contains a write set
- Includes one or more PutItem, UpdateItem, and DeleteItem operations across multiple tables

- **TransactGetItems**

- Contains a read set
- Includes one or more GetItem operations across multiple tables

- **If one operation fails, the entire transaction fails.**



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

62

You can use the DynamoDB transactional read and write APIs to manage complex business workflows. These workflows might require adding, updating, or deleting multiple items as a single, all-or-nothing operation. For example, a video game developer can ensure that players' profiles are updated correctly when they exchange items in a game or make in-game purchases.

New DynamoDB operations have been introduced for handling transactions:

- *TransactWriteItems*: A batch operation that contains a write set, with one or more PutItem, UpdateItem, and DeleteItem operations. TransactWriteItems can optionally check for prerequisite conditions that must be satisfied before updates are made. These conditions could involve the same or different items from the items in the write set. If any condition is not met, the transaction is rejected.
- *TransactGetItems*: A batch operation that contains a read set, with one or more GetItem operations. If a TransactGetItems request is issued on an item that is part of an active write transaction, the read transaction is canceled. To get the previously committed value, you can use a standard read.

With transactional operations, if one operation fails, the entire transaction fails.

For more information about transactional operations, see “New – Amazon DynamoDB Transactions” at <https://aws.amazon.com/blogs/aws/new-amazon-dynamodb-transactions/>.

Section 9 key takeaways



- DynamoDB has four basic operations:
 - Control: Create and manage tables
 - Data: Create, read, update, delete data
 - Batch: Read and write multiple items
 - Transaction: Coordinated, all-or-nothing change to multiple items

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

63

The following are the key takeaways from this section of the module:

There are four basic DynamoDB operations:

- Control: Create and manage tables
- Data: Create, read, update, delete data
- Batch: Read and write multiple items
- Transaction: Coordinated, all-or-nothing change to multiple items

Lab 5.1: Working with Amazon DynamoDB



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

64

You will now complete Lab 5.1: Working with Amazon DynamoDB.

Lab: Tasks

1. Preparing the lab
2. Creating a DynamoDB table by using the SDK for Python
3. Working with DynamoDB data – Understanding DynamoDB condition expressions
4. Adding and modifying a single item by using the SDK
5. Adding multiple items by using the SDK and batch processing
6. Querying the table by using the SDK
7. Adding a global secondary index to the table

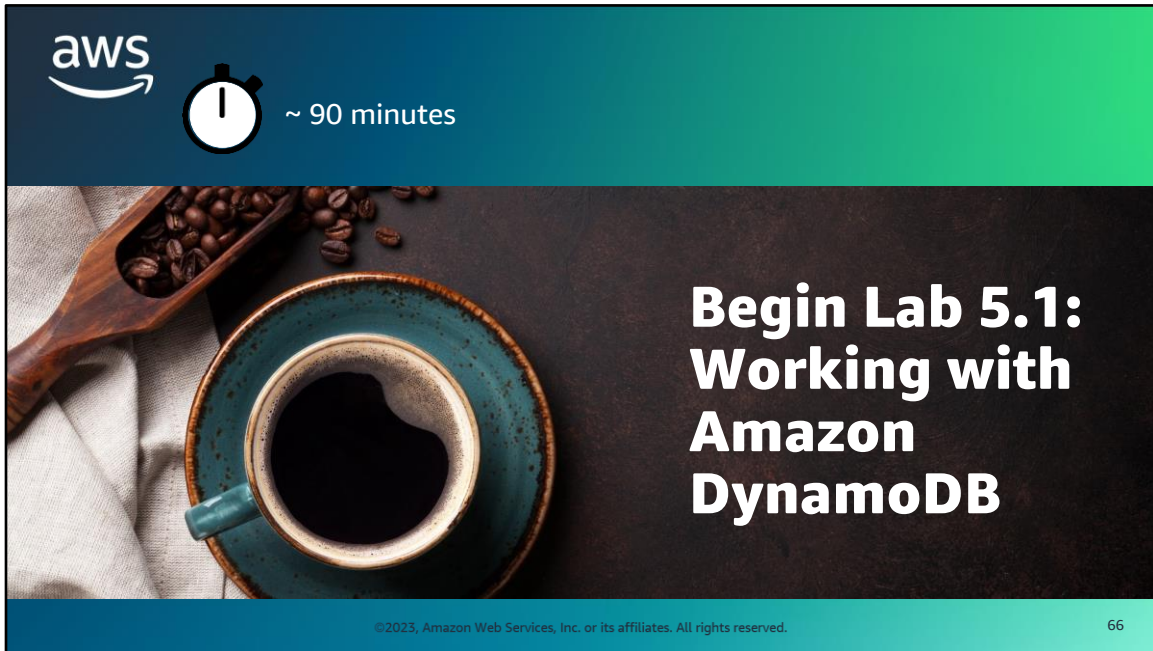


©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

65

In this lab, you will complete the following tasks:

1. Preparing the lab
2. Creating a DynamoDB table by using the SDK for Python
3. Working with DynamoDB data – Understanding DynamoDB condition expressions
4. Adding and modifying a single item by using the SDK
5. Adding multiple items by using the SDK and batch processing
6. Querying the table by using the SDK
7. Adding a global secondary index to the table



The image is a promotional banner for an AWS lab. It features a dark teal background with a green gradient at the top. In the top left corner, the AWS logo is displayed next to a clock icon and the text '~ 90 minutes'. The central part of the banner shows a top-down view of a wooden coffee scoop filled with coffee beans, a few scattered beans, and a teal ceramic coffee cup filled with black coffee. On the right side, the text 'Begin Lab 5.1: Working with Amazon DynamoDB' is written in a large, white, sans-serif font. At the bottom, a thin teal bar contains the copyright notice '©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.' on the left and the number '66' on the right.

aws ~ 90 minutes

**Begin Lab 5.1:
Working with
Amazon
DynamoDB**

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved. 66

It is now time to start the lab.

Lab debrief: Key takeaways

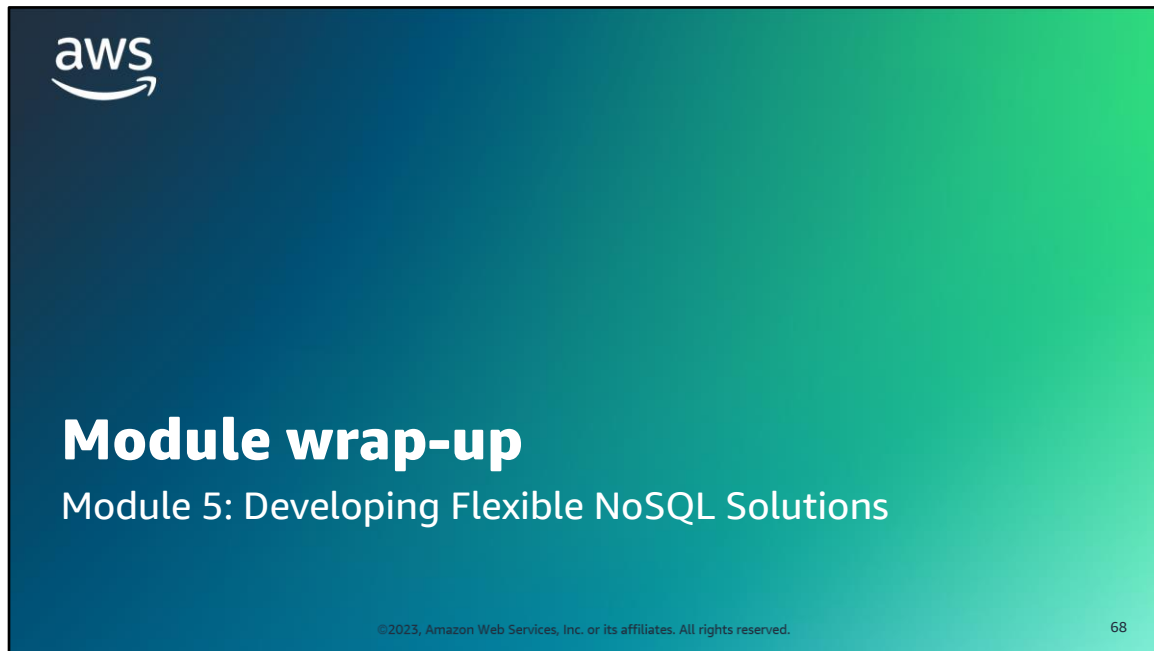


aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

67

Your educator might choose to lead a conversation about the key takeaways from this lab after you have completed it.



It's now time to review the module and wrap up with a knowledge check and discussion of a practice certification exam question.

Module summary

In summary, in this module, you learned how to do the following:

- Recognize different database options on AWS
- Identify the features of DynamoDB
- Describe the components of DynamoDB
- Explain how DynamoDB uses partitions
- Indicate how indexes are used with DynamoDB
- Describe how DynamoDB keeps data consistent
- Recognize when streaming and global tables are used
- Explain the backup and restore process
- Develop with DynamoDB by using the AWS SDKs



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

69

In summary, in this module, you learned how to do the following:

- Recognize different database options on AWS
- Identify the features of DynamoDB
- Describe the components of DynamoDB
- Explain how DynamoDB uses partitions
- Indicate how indexes are used with DynamoDB
- Describe how DynamoDB keeps data consistent
- Recognize when streaming and global tables are used
- Explain the backup and restore process
- Develop with DynamoDB by using the AWS SDKs

Complete the knowledge check



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

70

It is now time to complete the knowledge check for this module.

Sample exam question

A developer is creating a solution that will use an Amazon DynamoDB table. Anytime the table is updated, the changes must be sent to AWS Lambda. Lambda will then send a message to the Operations team when certain values exist.

How could the developer modify the table to create this solution?

Identify the key words and phrases before continuing.

The following are the key words and phrases:

- **Amazon DynamoDB**
- **Updated, the changes must be sent**



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

71

It is important to fully understand the scenario and question being asked before even reading the answer choices. Find the keywords in this scenario and question that will help you find the correct answer.

Sample exam question: Response choices

A developer is creating a solution that will use an **Amazon DynamoDB** table. Anytime the table is **updated, the changes must be sent** to AWS Lambda. Lambda will then send a message to the Operations team when certain values exist.

How could the developer modify the table to create this solution?

Choice	Response
A	Create a global secondary index to speed up queries so the developer's code can get faster results.
B	Create a local secondary index to speed up queries so the developer's code can get faster results.
C	Enable DynamoDB Streams on the table.
D	None of the above

 ©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved. 72

Now that we have bolded the keywords in this scenario, let us look at the answers.

Sample exam question: Answer

The correct answer is C.

Choice	Response
A	Create a global secondary index to speed up queries so the developer's code can get faster results.
B	Create a local secondary index to speed up queries so the developer's code can get faster results.
C	Enable DynamoDB Streams on the table.
D	None of the above

 ©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved. 73

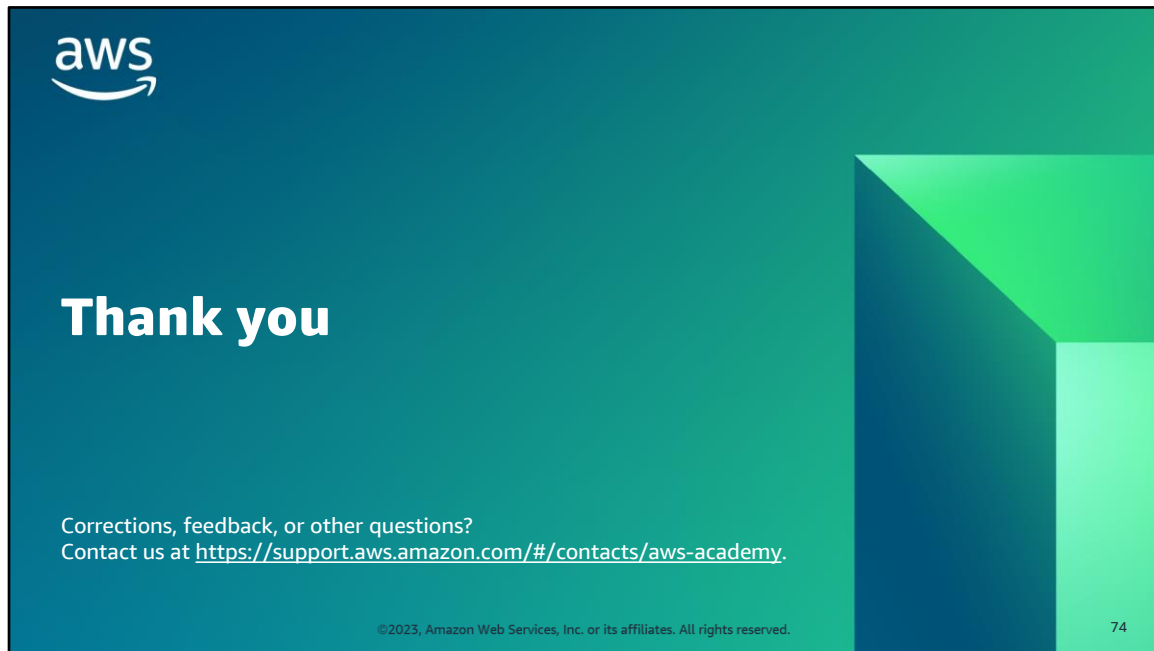
Look at the answer choices and rule them out based on the keywords that were previously highlighted.

The correct answer is C. Enable DynamoDB Streams on the table.

When DynamoDB Streams is enabled, the table will emit one of the four options to the stream:

1. **Keys only:** Only the key attributes of the modified item
2. **New image:** The entire item, as it appears after it was modified
3. **Old image:** The entire item, as it appeared before it was modified
4. **New and old images:** Both the new and the old images of the item

The developer could select the **New Image** option, which writes update information to the stream. This Stream could then be associated with the target Lambda function. The Lambda function would receive update information that it could use to determine whether the Operations team must be notified.



Thank you for completing this module.